Bazele inteligenței artificiale

Laborator – Grigore Ionuț

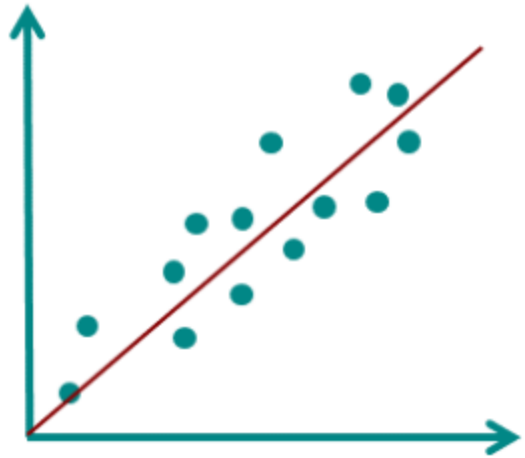
Laboratory contents

1. Introduction to Python
2. Linear models → **today's lab**
3. Nonparametric models
4. Decision trees. Ensemble methods
5. Dimensionality reduction
6. Clustering
7. Test 1
8. Introduction to PyTorch
9. Multilayer perceptrons
10. Convolutional neural networks
11. Recurrent neural networks
12. Attention mechanisms
13. Test 2
14. Recoveries

Content

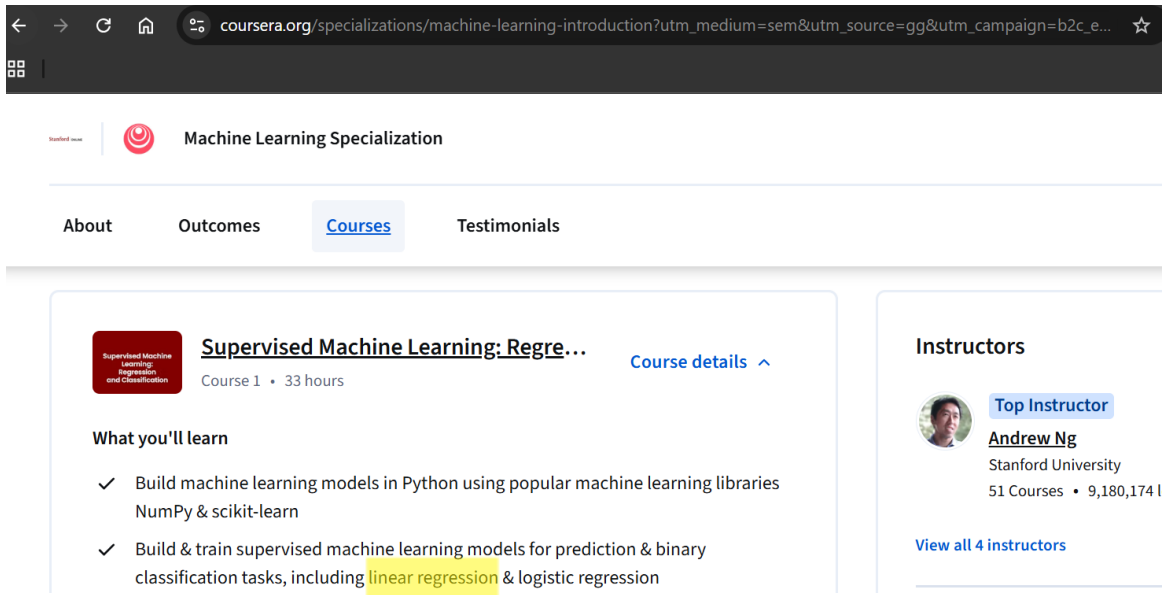
- Linear regression
- Gradient descent
- Polynomial regression
- Regularized linear models
- Logistic regression
- Softmax regression

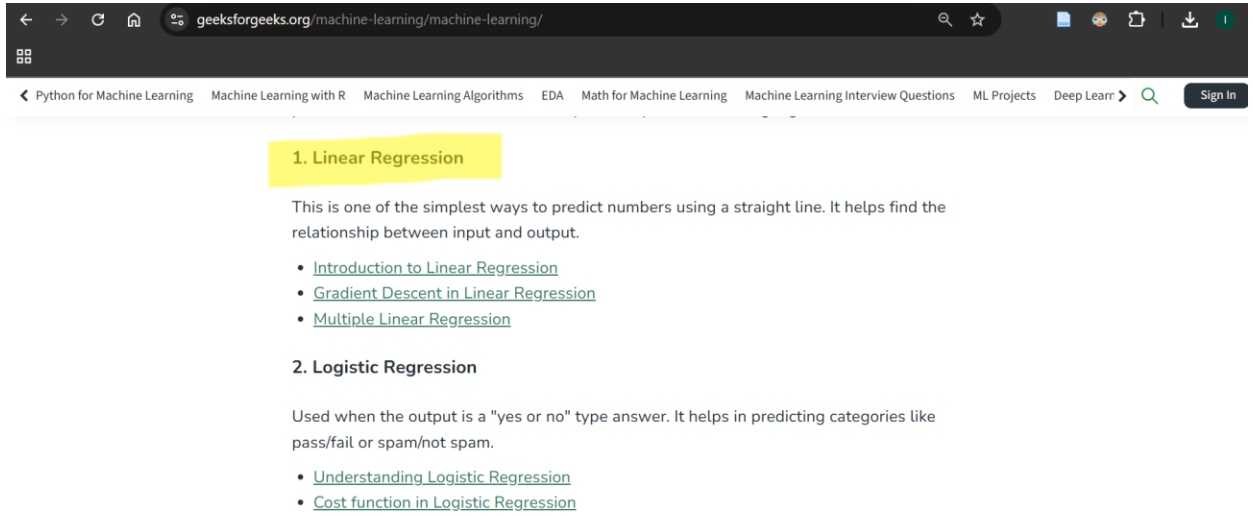
1) Linear regression

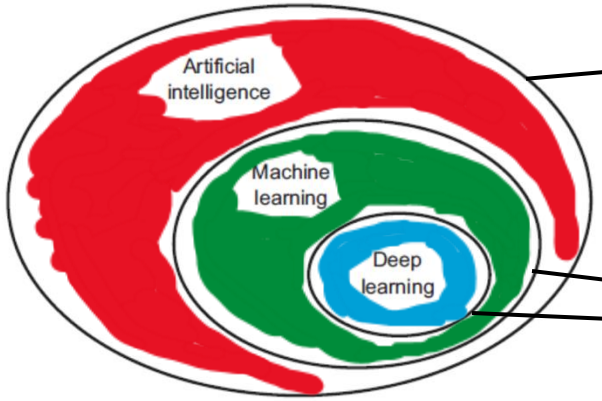


- Most ML courses begin with **linear regression**, as it introduces the essential concepts of learning from data.
- Linear regression = **the “hello world” of ML**





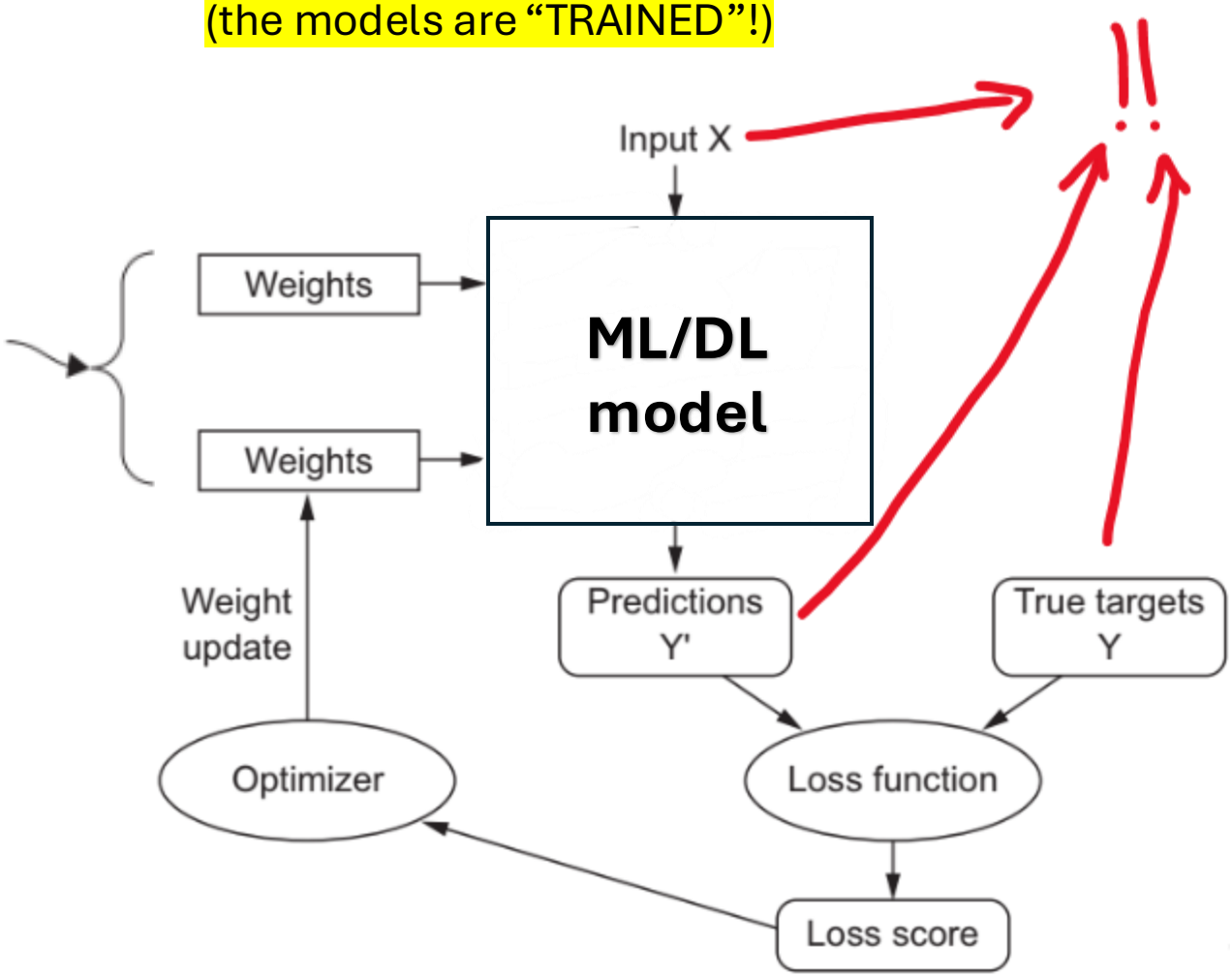




does not involve learning from data !

learn from data to make predictions or decisions!
(the models are “TRAINED”!)

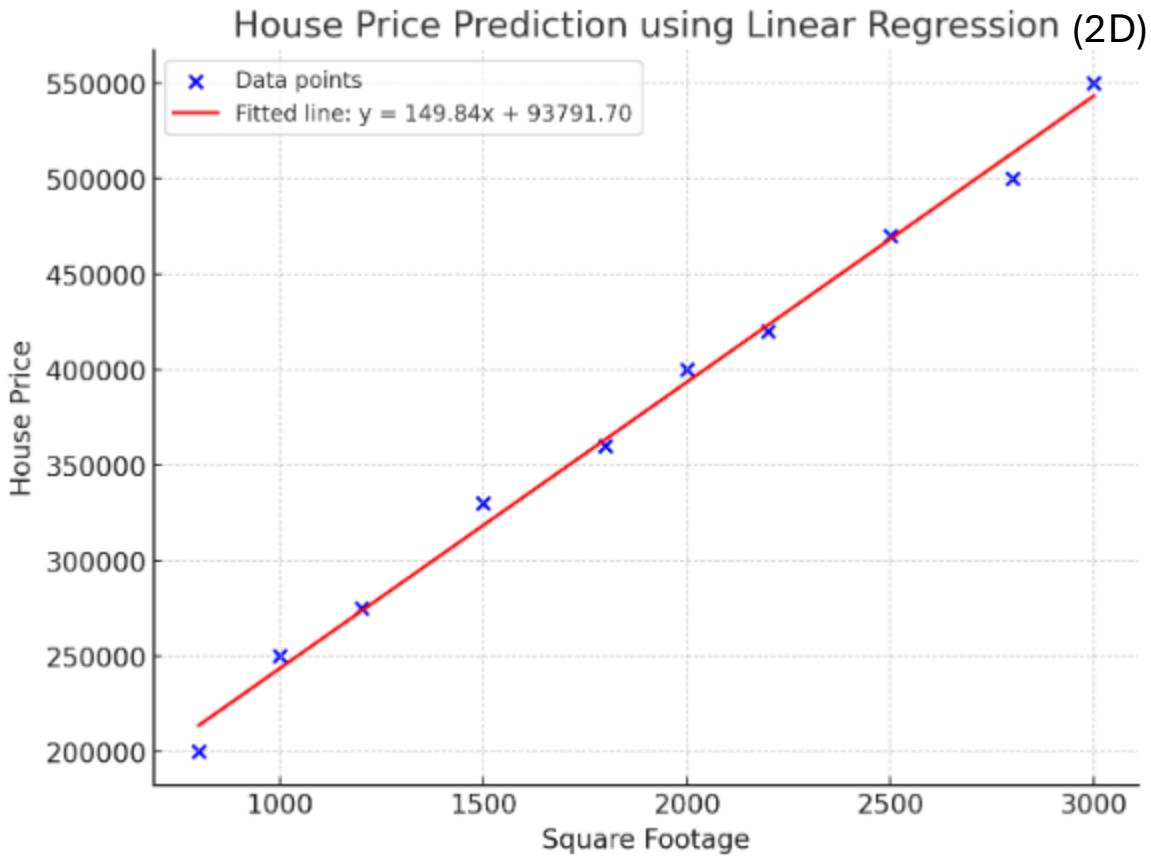
Goal: finding the right values for these weights



A simple example to understand the problem

House Price Prediction Data

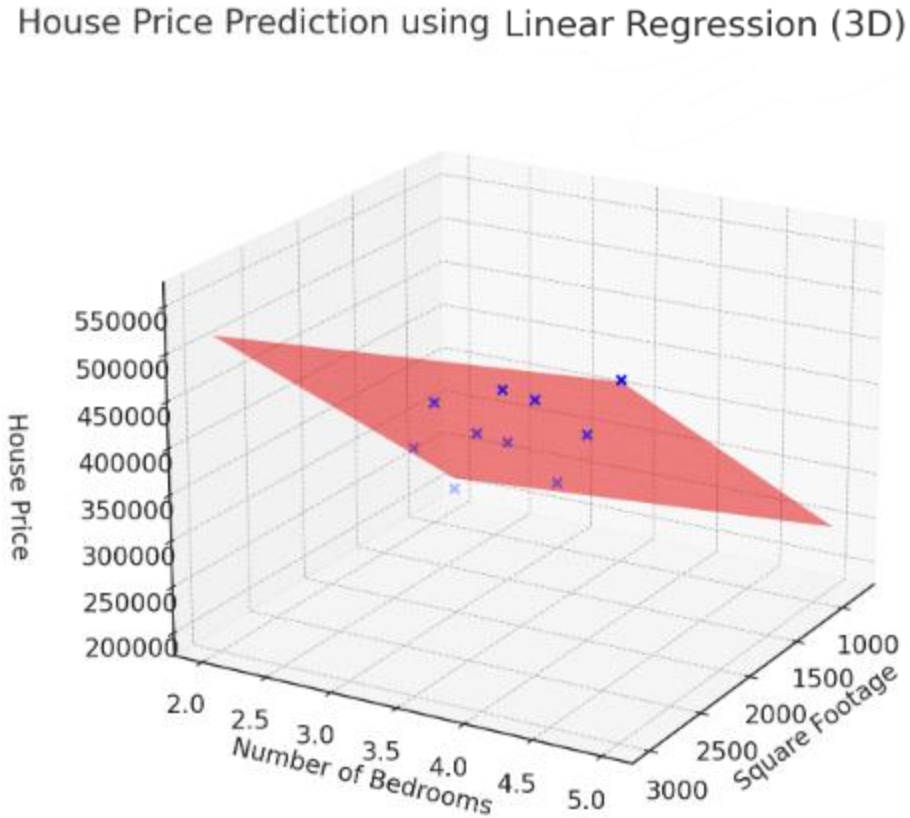
	SquareFootage	HousePrice ($)
1	800	200000
2	1000	250000
3	1200	275000
4	1500	330000
5	1800	360000
6	2000	400000
7	2200	420000
8	2500	470000
9	2800	500000
10	3000	550000



$$\text{House Price} = 149.84 \times \text{Square Footage} + 93791.70$$

House Price Prediction Data With Additional Feature

	SquareFootage	NumBedrooms	HousePrice ($)
1	800	2	200000
2	1000	3	250000
3	1200	2	275000
4	1500	3	330000
5	1800	3	360000
6	2000	4	400000
7	2200	3	420000
8	2500	4	470000
9	2800	4	500000
10	3000	5	550000



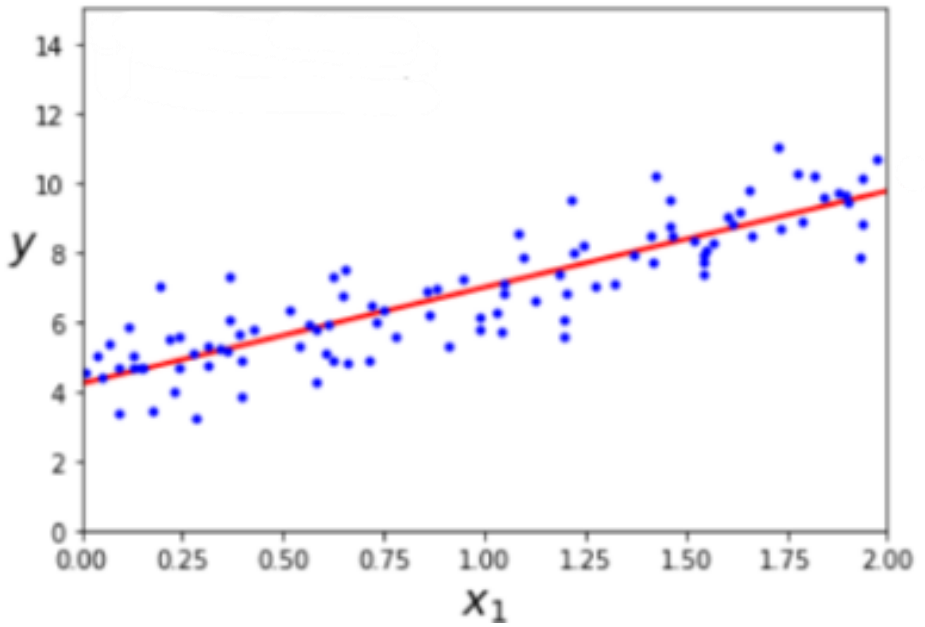
$$\text{House Price} = 137.99 \times \text{Square Footage} + 10800.78 \times \text{Num Bedrooms} + 80439.45$$

UDL Interactive Figures
see here 2.2 Least squares loss

➢ the linear regression model is expressed as:

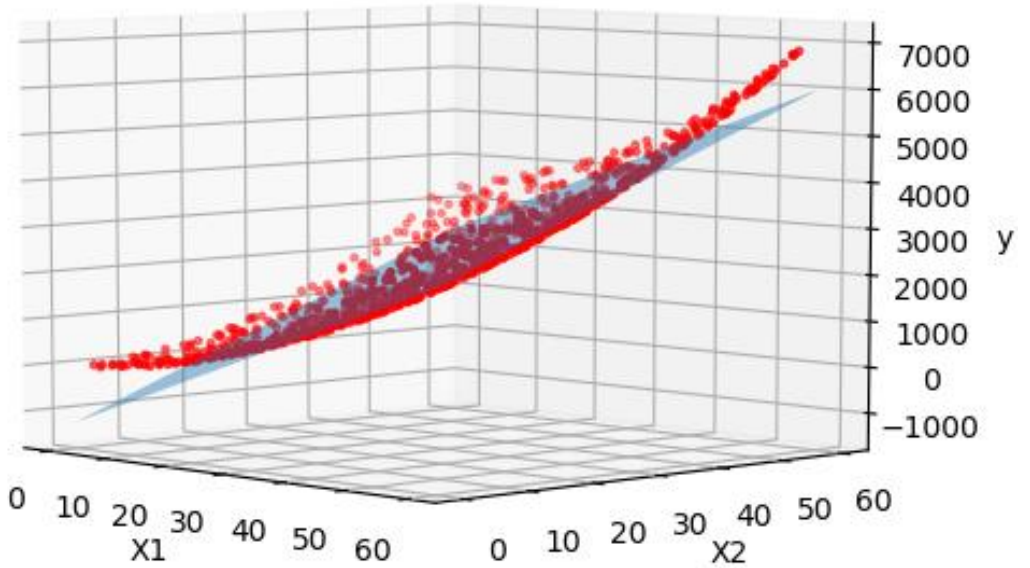
$$\hat{y} = w_0 + w_1 x_1 + \cdots + w_d x_d.$$

$\hat{y}$: predicted value; w_0: bias; w_1: feature weight; x_1: feature; w_d: feature weight; x_d: feature



d=2

$\hat{y} = w_0 + w_1 x_1$ → equation of a line



d=3

$\hat{y} = w_0 + w_1 x_1 + w_2 x_2$ → equation of a plane

NOTE: We cannot plot more than 3 dimensions ! ☺ (actually we can but it's more tricky)

- **linear regression** method predicts a real-valued output (*label*/*target*) $y \in \mathbb{R}$, given a set of real-valued inputs (*features*) $x_1, x_2, \ldots, x_d$;

- to develop a model for predicting $y \in \mathbb{R}$, we need to obtain a *dataset* consisting of known outputs for corresponding inputs;

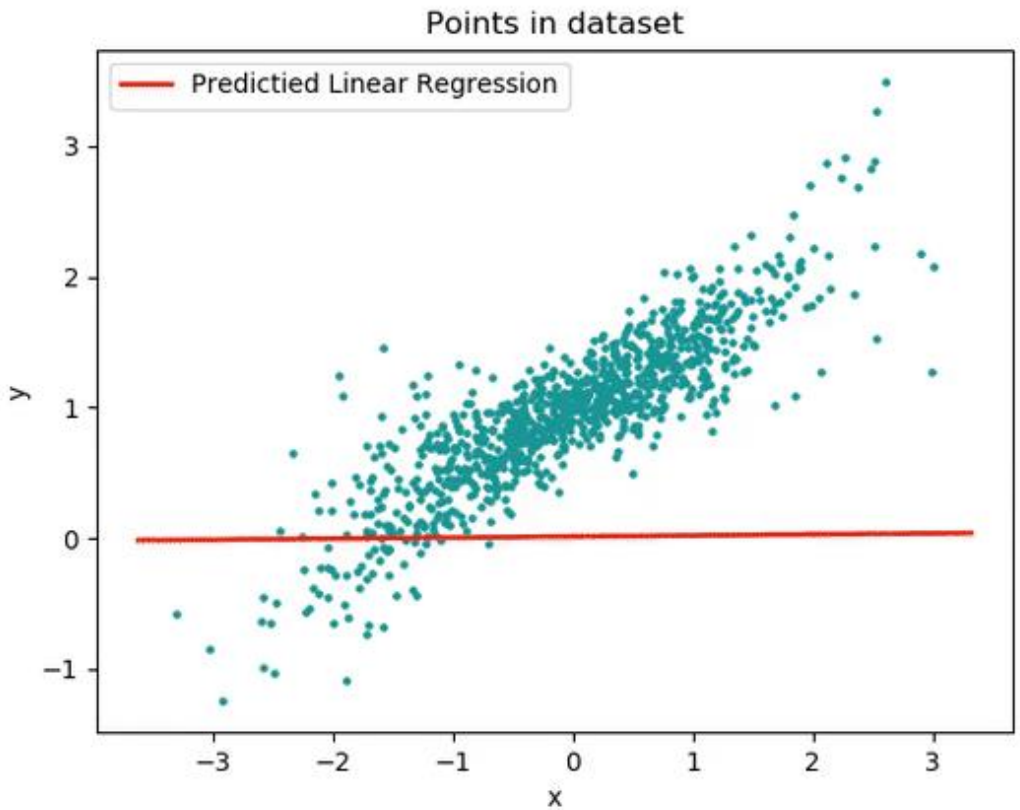
training dataset – each row is called: ***example*** or ***data point*** or ***sample***

- the linear regression model is expressed as:

$$\hat{y} = w_0 + w_1 \overset{\text{feature}}{\overset{\downarrow}{x_1}} + \cdots + w_d \overset{\text{feature}}{\overset{\downarrow}{x_d}}.$$

$\hat{y}$: predicted value; w_0: *bias*; w_1: feature weight; w_d: feature weight

- the weights determine the influence of each feature on our prediction;
- given a dataset, our goal is to choose the weights $w_1, w_2, \ldots, w_d$ and the bias w_0 such that, on average, the predictions $\hat{y}$ made by our model *best fit* the true labels y observed in the data;



$$\boldsymbol{X} = \begin{bmatrix} 1 & x_1^{(1)} & x_2^{(1)} & \dots & x_d^{(1)} \\ 1 & x_1^{(2)} & x_2^{(2)} & \dots & x_d^{(2)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(n)} & x_2^{(n)} & \dots & x_d^{(n)} \end{bmatrix} = \begin{bmatrix} \boldsymbol{x}^{(1)\top} \\ \boldsymbol{x}^{(2)\top} \\ \vdots \\ \boldsymbol{x}^{(n)\top} \end{bmatrix}$$

$\in \mathbb{R}^{n \times (d+1)}$

One input-output pair or one training **example/sample**

$$\boldsymbol{y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n)} \end{bmatrix}$$

features of the training dataset

corresponding (known) labels

training dataset

$\hat{\boldsymbol{y}} = \boldsymbol{X}\boldsymbol{w}$

linear regression model

$$\hat{\boldsymbol{y}} = \begin{bmatrix} \hat{y}^{(1)} \\ \hat{y}^{(2)} \\ \vdots \\ \hat{y}^{(n)} \end{bmatrix}$$

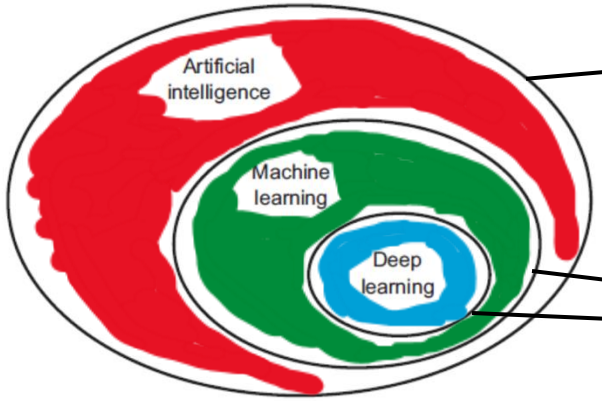
predictions

We want them to be the same!

$$\boldsymbol{y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(n)} \end{bmatrix}$$

true (known) labels

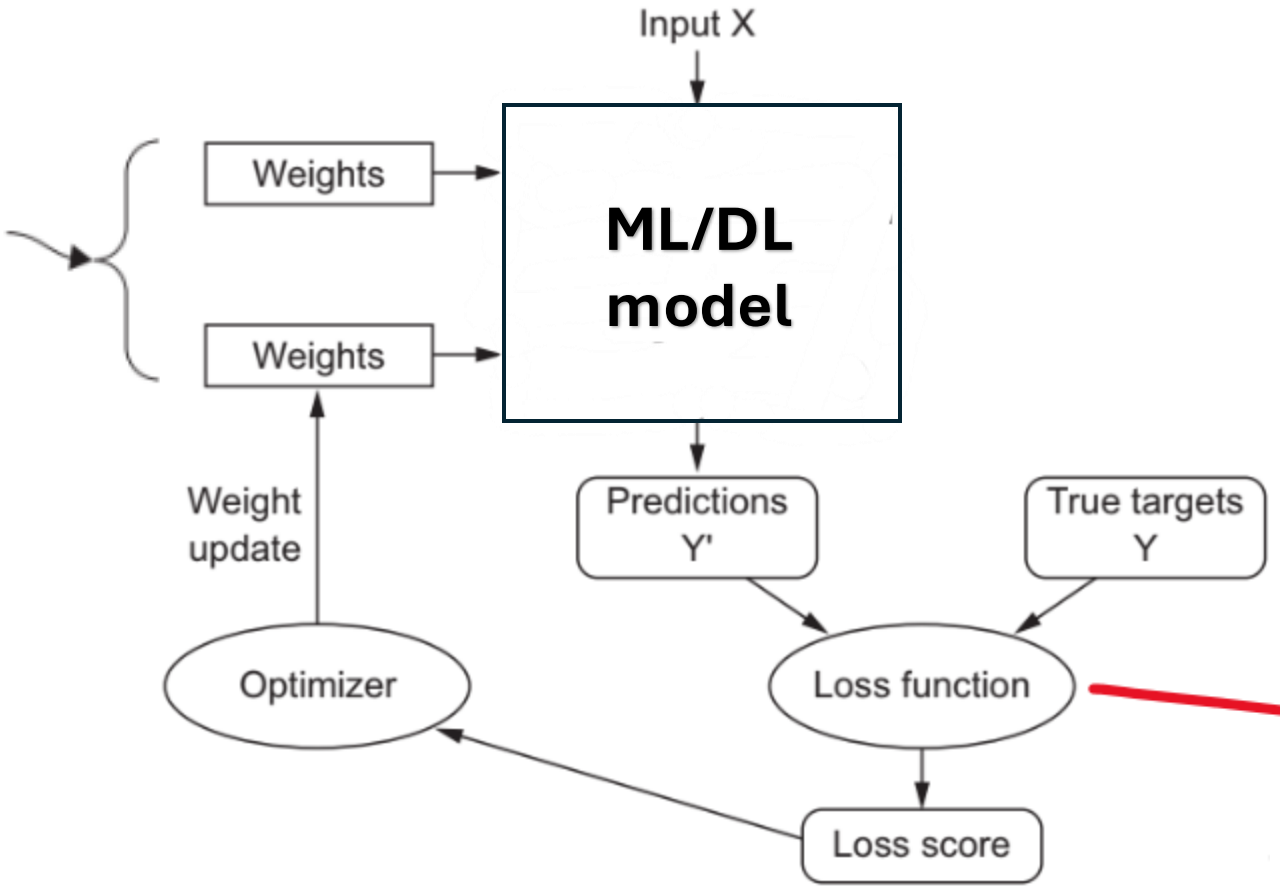
Loss function



does not involve learning from data !

learn from data to make predictions or decisions!
(the models are “TRAINED”!)

Goal: finding the right values for these weights





- we must create a formula for the loss function !

- the quality measure for a certain ML model is given by the **loss function** which quantifies the distance between the *real* and *predicted* value of the target (smaller values are better and perfect predictions have a loss of 0)

- the most popular loss function in regression problems is the *squared error*
- when the model prediction for an example i is $\hat{y}^{(i)}$ and the corresponding true label is $y^{(i)}$, the squared error is given by:

$$l^{(i)}(\boldsymbol{w}) = \frac{1}{2}(\hat{y}^{(i)} - y^{(i)})^2.$$

NOTE: the constant ½ will cancel out when we take the derivative of the loss !

- to measure the quality of a model on the entire dataset of n examples, we simply average (or equivalently, sum) the losses on the training set:

$$\mathcal{L}(\boldsymbol{w}) = \frac{1}{n}\sum_{i=1}^{n} l^{(i)}(\boldsymbol{w}) = \frac{1}{n}\sum_{i=1}^{n} \frac{1}{2}(\hat{y}^{(i)} - y^{(i)})^2 = \frac{1}{n}\sum_{i=1}^{n} \frac{1}{2}(\boldsymbol{w}^\top \boldsymbol{x}^{(i)} - y^{(i)})^2.$$

mean squared error loss function

A probabilistic interpretation of linear regression

(the ugly part from the course with probabilities ☺)



- we assume that observations arise from noisy observations, where the noise is normally distributed as follows:

$$y = \boldsymbol{w}^\top \boldsymbol{x} + \epsilon, \qquad \text{where } \epsilon \sim \mathcal{N}(0, \sigma^2)$$

  - **Measurement errors**: Imperfections in the instruments or processes used to collect data.
  - **External factors**: Influences from environmental variables that aren't modeled.
  - etc.

Example:

Imagine you're collecting data on house prices and you expect a linear relationship between the size of a house and its price. However, in reality, other factors (like the neighborhood, condition of the house, economic conditions) introduce noise into the data, so the actual price of a house deviates from the expected price based on its size.

- to refresh our memory, the probability density of a *normal distribution* $\mathcal{N}(\mu, \sigma^2)$ with *mean* μ and *variance* σ^2 (standard deviation σ) is given as:



$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2\sigma^2}(x-\mu)^2\right).$$

- thus, we can now write the *likelihood* of seeing a particular $y^{(i)}$ for a given $\boldsymbol{x}^{(i)}$ as:

$$p(y^{(i)}|\boldsymbol{x}^{(i)}, \boldsymbol{w}) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2\sigma^2}\left(y^{(i)} - \boldsymbol{w}^\top \boldsymbol{x}^{(i)}\right)^2\right).$$

residual error follows a normal distribution with mean 0 and variance σ^2

the *likelihood* of seeing a particular $y^{(i)}$ for a given $\boldsymbol{x}^{(i)}$ ⟺ the *likelihood* of appearing the ***residual error***

- The smaller the residual $y^{(i)} - \boldsymbol{w}^\top \boldsymbol{x}^{(i)}$, the higher the likelihood of observing that data point
- The higher the residual $y^{(i)} - \boldsymbol{w}^\top \boldsymbol{x}^{(i)}$, the smaller the likelihood of observing that data point



- now, according to the principle of *maximum likelihood*, the best values of weights $\boldsymbol{w}$ are those that maximize the likelihood of the entire dataset:

we want to maximize this!

$$p(\boldsymbol{y}|\boldsymbol{X}, \boldsymbol{w}) = \prod_{i=1}^{n} p(y^{(i)}|\boldsymbol{x}^{(i)}, \boldsymbol{w}).$$

mathematical proof on the next slide

maximizing the product of many exponentials might be difficult, so we instead minimize the following:

$$-\log p(\boldsymbol{y}|\boldsymbol{X}, \boldsymbol{w}) = \sum_{i=1}^{n} \left[\frac{1}{2} \log(2\pi\sigma^2) + \frac{1}{2\sigma^2} (y^{(i)} - \boldsymbol{w}^\top \boldsymbol{x}^{(i)})^2 \right]$$

we want to minimize this!

negative log-likelihood



$$p(\mathbf{y}|\mathbf{X},\mathbf{w}) = \prod_{i=1}^{n} p(y^{(i)}|\mathbf{x}^{(i)},\mathbf{w}) \rightarrow p(\mathbf{y}|\mathbf{X},\mathbf{w}) = \prod_{i=1}^{n} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y^{(i)} - \mathbf{w}^\top \mathbf{x}^{(i)})^2}{2\sigma^2}\right) \Big| \log \rightarrow$$

$$\rightarrow \log p(\mathbf{y}|\mathbf{X},\mathbf{w}) = \sum_{i=1}^{n} \log\left(\frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y^{(i)} - \mathbf{w}^\top \mathbf{x}^{(i)})^2}{2\sigma^2}\right)\right)$$ (We used log(ab) = log(a) + log(b), thus the product became sum) →

$$\rightarrow \log p(\mathbf{y}|\mathbf{X},\mathbf{w}) = \sum_{i=1}^{n} \left[\log\left(\frac{1}{\sqrt{2\pi\sigma^2}}\right) + \log\left(\exp\left(-\frac{(y^{(i)} - \mathbf{w}^\top \mathbf{x}^{(i)})^2}{2\sigma^2}\right)\right)\right]$$ (We used the same formula as above ☺) →

$$\rightarrow \log p(\mathbf{y}|\mathbf{X},\mathbf{w}) = \sum_{i=1}^{n} \left[-\frac{1}{2}\log(2\pi\sigma^2) - \frac{(y^{(i)} - \mathbf{w}^\top \mathbf{x}^{(i)})^2}{2\sigma^2}\right] \rightarrow$$

$$\rightarrow -\log p(\mathbf{y}|\mathbf{X},\mathbf{w}) = \sum_{i=1}^{n} \left[\frac{1}{2}\log(2\pi\sigma^2) + \frac{(y^{(i)} - \mathbf{w}^\top \mathbf{x}^{(i)})^2}{2\sigma^2}\right]$$, q.e.d.

OPTIONAL

$$-\log p(\boldsymbol{y}|\boldsymbol{X}, \boldsymbol{w}) = \sum_{i=1}^{n} \left[\frac{1}{2} \log(2\pi\sigma^2) + \frac{1}{2\sigma^2} (y^{(i)} - \boldsymbol{w}^\top \boldsymbol{x}^{(i)})^2 \right]$$

we want to minimize this!

fixed, so we ignore it

fixed, so we ignore it

- as a consequence, negative log-likelihood is given as:

$$\sum_{i=1}^{n} \frac{1}{2} (y^{(i)} - \boldsymbol{w}^\top \boldsymbol{x}^{(i)})^2 \quad \text{or} \quad \sum_{i=1}^{n} \frac{1}{2} (y^{(i)} - \hat{y}^{(i)})^2 \quad \left(\text{because } \hat{y}^{(i)} = \boldsymbol{w}^\top \boldsymbol{x}^{(i)}\right)$$

we want to minimize this!

We will see why we keep this ½ factor

OPTIONAL

TL;DR: What Did We Learn?

• Our data always has **noise** (errors, missing factors, randomness).

• We assume this noise follows a **normal distribution** (see **Central Limit Theorem**).

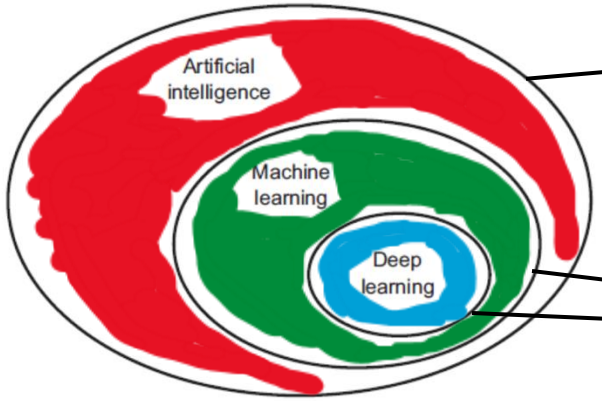
• We want to find weights w that make our model's predictions **fit the data as well as possible**.

• This is done by **Maximum Likelihood Estimation (MLE)** = finding the parameters that make the data most "likely."

• **In practice** → this means minimizing the **squared differences** between predictions and true values.



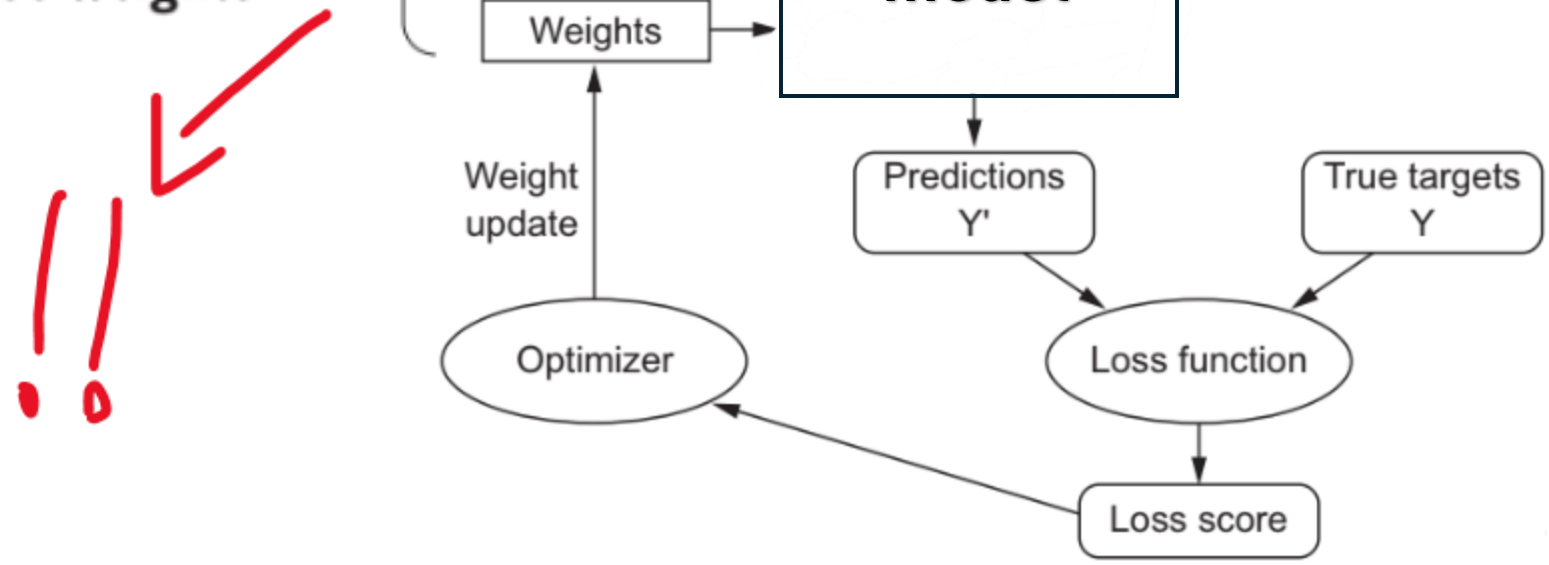
How do we find weights w?



does not involve learning from data !

learn from data to make predictions or decisions!
(the models are “TRAINED”!)

Goal: finding the right values for these weights



- the mean squared error can be written as:

$$\mathcal{L}(\boldsymbol{w}) = \frac{1}{2n}(\boldsymbol{X}\boldsymbol{w} - \boldsymbol{y})^\top(\boldsymbol{X}\boldsymbol{w} - \boldsymbol{y})$$

- now we want to find the parameters $\boldsymbol{w}$ that minimizes the total loss across all training samples, $\mathcal{L}(\boldsymbol{w})$:

$$\boldsymbol{w}^* = \underset{\boldsymbol{w}}{\operatorname{argmin}}\, \mathcal{L}(\boldsymbol{w})$$

- recall from first year calculus that, in order to find the minimum of the function $\mathcal{L}(\boldsymbol{w})$, we must compute its gradient, equal it with $\mathbf{0}^\top$, and extract the value of the minimum:



$$\begin{aligned}
\nabla_{\boldsymbol{w}}\mathcal{L}(\boldsymbol{w}) &= \nabla_{\boldsymbol{w}}\left(\frac{1}{2n}(\boldsymbol{X}\boldsymbol{w} - \boldsymbol{y})^\top(\boldsymbol{X}\boldsymbol{w} - \boldsymbol{y})\right) \\
&= \frac{1}{2n}\nabla_{\boldsymbol{w}}\left(\boldsymbol{w}^\top\boldsymbol{X}^\top\boldsymbol{X}\boldsymbol{w} - 2\boldsymbol{y}^\top\boldsymbol{X}\boldsymbol{w} + \boldsymbol{y}^\top\boldsymbol{y}\right) \\
&= \frac{1}{n}(\boldsymbol{X}^\top\boldsymbol{X}\boldsymbol{w} - \boldsymbol{X}^\top\boldsymbol{y}).
\end{aligned}$$

- from $\nabla_{\boldsymbol{w}}\mathcal{L}(\boldsymbol{w}) = \mathbf{0}^\top$, we obtain:

$$\begin{aligned}
\nabla_{\boldsymbol{w}}\mathcal{L}(\boldsymbol{w}) = \mathbf{0}^\top &\Leftrightarrow \boldsymbol{X}^\top\boldsymbol{X}\boldsymbol{w} - \boldsymbol{X}^\top\boldsymbol{y} = \mathbf{0}^\top \\
&\Leftrightarrow \boldsymbol{X}^\top\boldsymbol{X}\boldsymbol{w} = \boldsymbol{X}^\top\boldsymbol{y} \\
&\Leftrightarrow \boldsymbol{w} = (\boldsymbol{X}^\top\boldsymbol{X})^{-1}\boldsymbol{X}^\top\boldsymbol{y}.
\end{aligned}$$

- thus, for linear regression, we have the analytic (closed-form) solution:

$$\boldsymbol{w}^* = (\boldsymbol{X}^\top\boldsymbol{X})^{-1}\boldsymbol{X}^\top\boldsymbol{y}.$$



Code part

Method 1. Implement manually

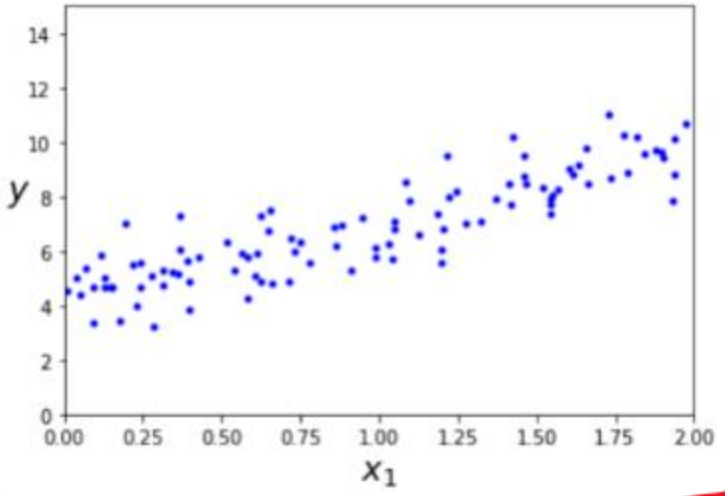
- generate data ($y = 4 + 3x_1 +$ *Gaussian noise*) and compute w^* using normal equation;

```
X = 2 * np.random.rand(100, 1) # d=1
y = 4 + 3 * X + np.random.randn(100, 1)

X_1 = np.c_[np.ones((100, 1)), X] # add x0 = 1 to each instance
w_star = np.linalg.inv(X_1.T.dot(X_1)).dot(X_1.T).dot(y)

w_star
```

```
array([[4.21509616],
       [2.77011339]])
```



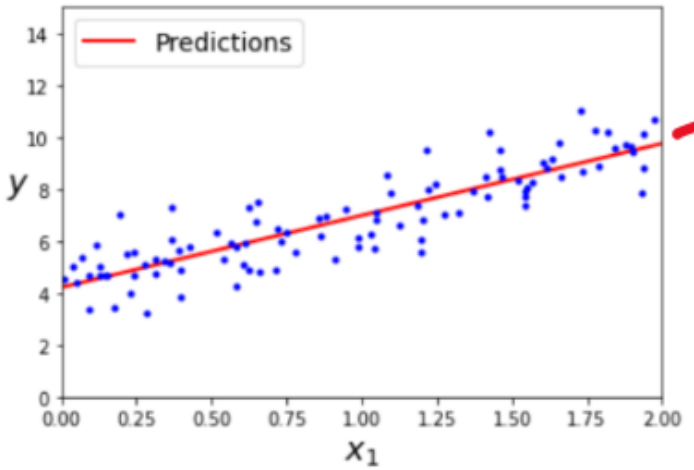
The equation of the red line is:

4.215 + 2.770*x

- generate predictions using w^*;

```
X_new = np.array([[0], [2]])
X_new_1 = np.c_[np.ones((2, 1)), X_new]  # add x0 = 1 to each instance
y_predict = X_new_1.dot(w_star)
y_predict
```

```
array([[4.21509616],
       [9.75532293]])
```



Method 2. Use scikit-learn

1. *LinearRegression* class;
2. *fit()* method $\rightarrow$ fits the dataset X, y;
3. *coef_* $\rightarrow$ retrieves $w_1, \ldots, w_d$;
4. *intercept_* $\rightarrow$ retrieves w_0;
5. *predict()* $\rightarrow$ predictions on new data;

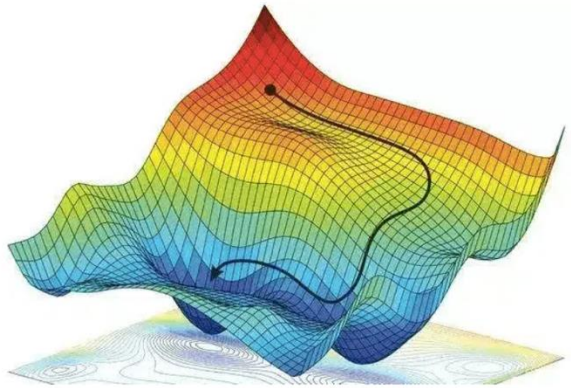
```
from sklearn.linear_model import LinearRegression

lin_reg = LinearRegression()
lin_reg.fit(X, y)
print("Bias: ", lin_reg.intercept_)
print("Parameter w1: ", lin_reg.coef_)

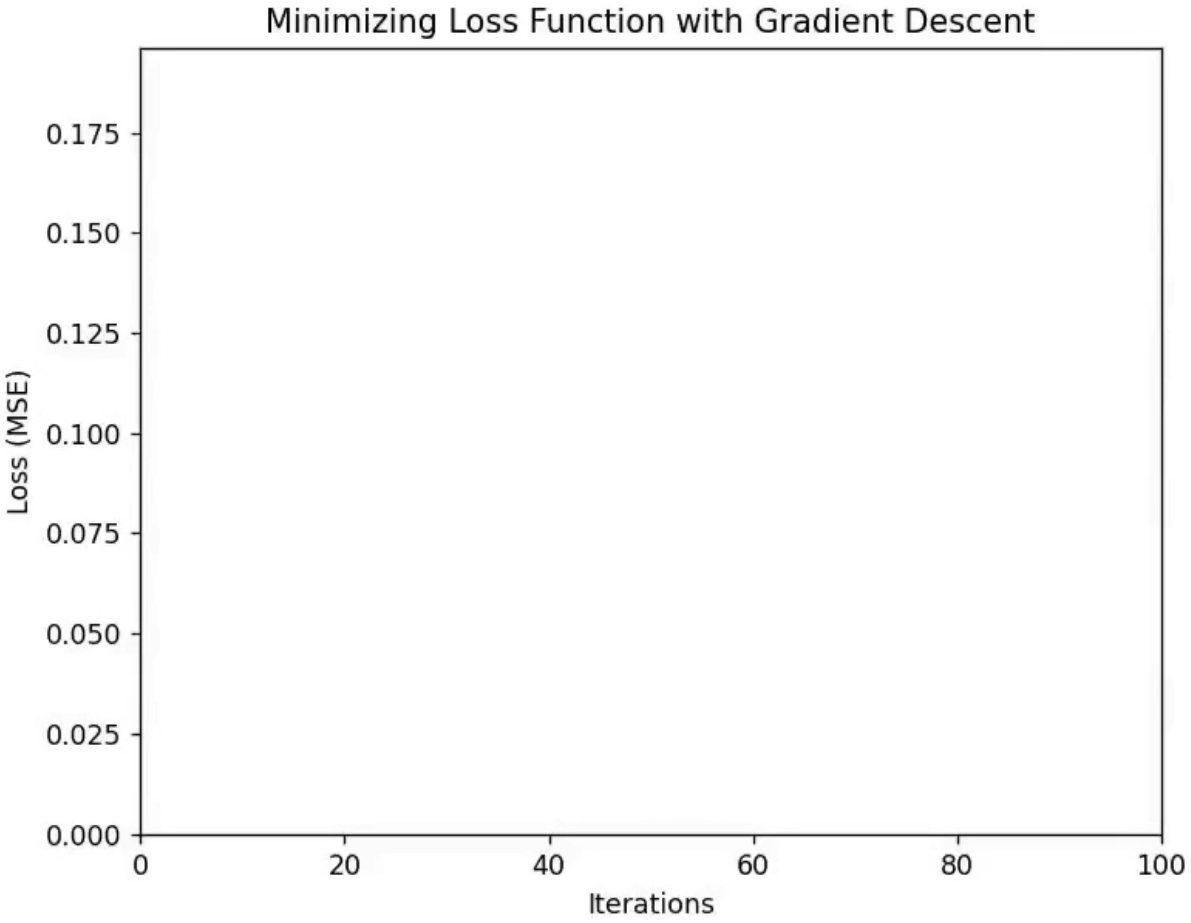
print("Predictions")
print(lin_reg.predict(X_new))
```

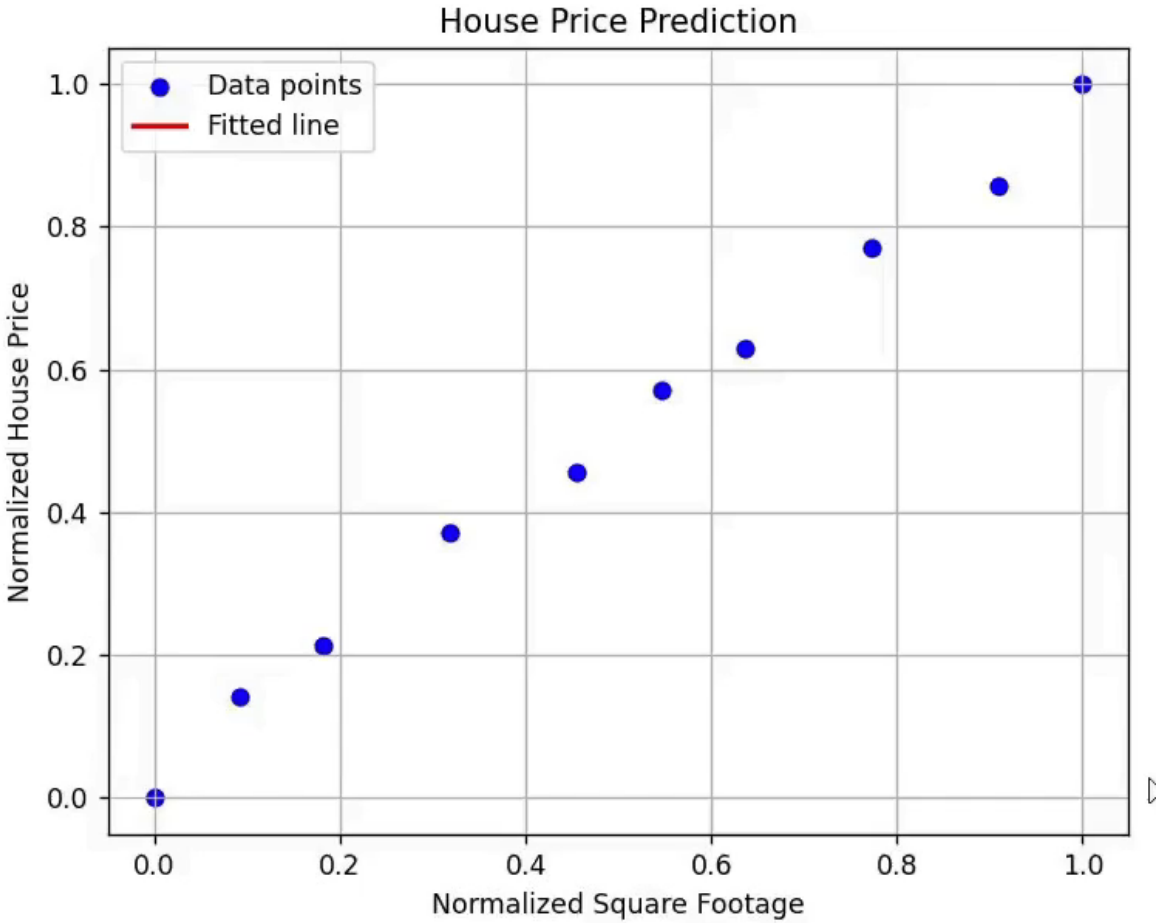
```
Bias:  [4.21509616]
Parameter w1:  [[2.77011339]]
Predictions
[[4.21509616]
 [9.75532293]]
```

2) Gradient descent



see here 2.4 Loss function: [UDL Interactive Figures]





- for linear regression, we have the analytic (closed-form) solution:

$$\boldsymbol{w}^* = (\boldsymbol{X}^\top \boldsymbol{X})^{-1} \boldsymbol{X}^\top \boldsymbol{y}.$$

- the normal equation computes the inverse of $\boldsymbol{X}^\top \boldsymbol{X}$, which is an $(d+1) \times (d+1)$ matrix
- the computational complexity of inverting such a matrix is typically about $\mathcal{O}(d^{2.4})$ to $\mathcal{O}(d^3)$, depending on the implementation

The normal equation gets very slow when the number of features grows large (e.g., 100, 000) !

$$\mathbf{X} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \qquad \mathbf{X} = \begin{bmatrix} 1 & 2 & 2 \\ 3 & 4 & 4 \\ 5 & 6 & 6 \\ 7 & 8 & 8 \end{bmatrix}$$

NOTES: when the matrix $\boldsymbol{X^TX}$ is not invertible (e.g. when $d > n$ or if one or more columns in X are identical), then we need to use single value decomposition (SVD) to compute the matrix $\boldsymbol{(X^TX)^{-1}X^T}$ which is called the *pseudoinverse* matrix of X ⟶ a complexity of about $\mathcal{O}(d^2)$

The SVD gets very slow when the number of features grows large (e.g., 100, 000) !

Solution: *gradient descent* !

- it updates the parameters iteratively in order to minimize the loss function;

- the steps of this algorithm are the following:
  1. initialize the values of the model weights at random;
  2. iteratively update the parameters:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}),$$

  where η is the learning rate and $\mathcal{L}(\cdot)$ is the mean squared error loss function.

NOTES:

- the value of the learning rate is manually pre-specified and not learned through model training;
- this type of parameters that are tunable but not updated in the training loop are called hyperparameters;



Input X



initially random numbers

Weights

Weights

ML/DL model

Weight update

$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w})$

"**alien**"

Predictions Y'

True targets Y

"**dog**"

Optimizer

Loss function

Loss score

2.8 - high



Input X



slightly better numbers

Weights

Weights

ML/DL model

Weight update

$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w})$

"**wolf**"

Predictions Y'

True targets Y

"**dog**"

Optimizer

Loss function

Loss score

0.9 - medium



Input X



very good numbers

Weights

Weights

ML/DL model

Weight update

$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w})$

"**dog**"

Predictions Y'

True targets Y

"**dog**"

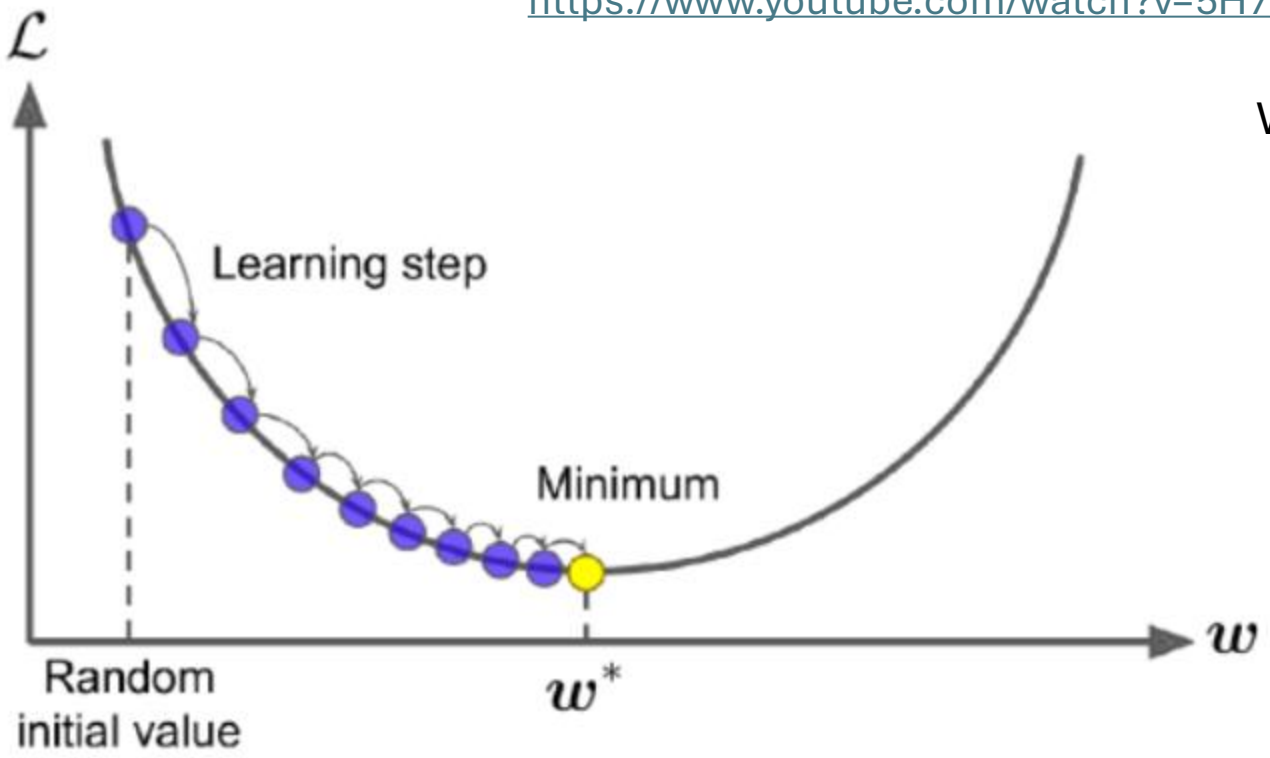
Optimizer

Loss function

Loss score

0.05 - low

https://www.youtube.com/watch?v=5H7M5Vd3-pk&list=PLpFsSf5Dm-pd5d3rjNtIXUHT-v7bdaEIe&index=12



Why this formula? $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w})$

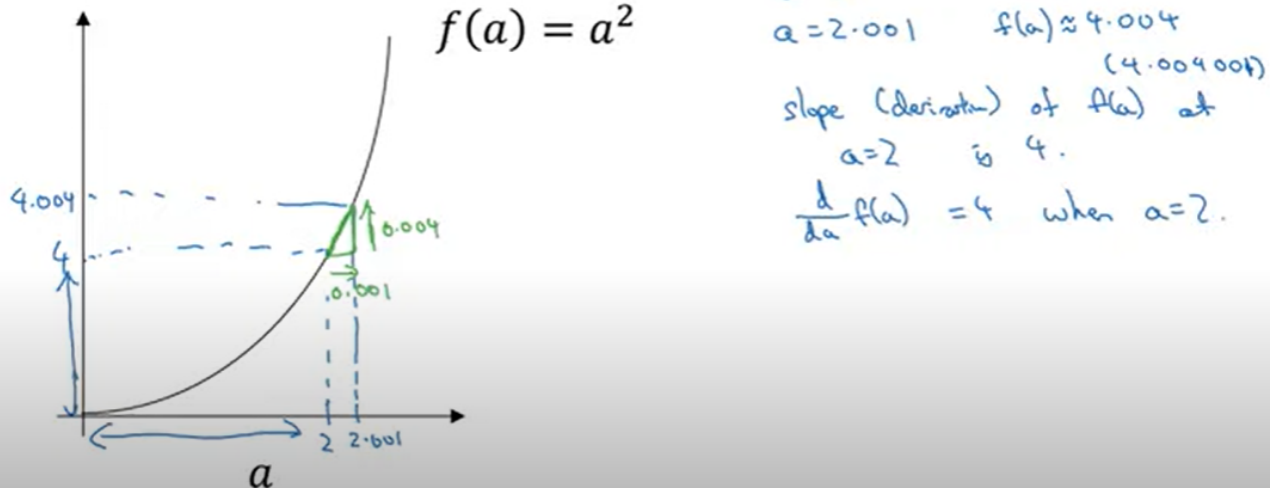
What is the derivative of a function?



$$f'(a) = \lim_{h \to 0} \frac{f(a+h) - f(a)}{h}$$

$$f'(a) = \lim_{x \to a} \frac{f(x) - f(a)}{x - a}$$

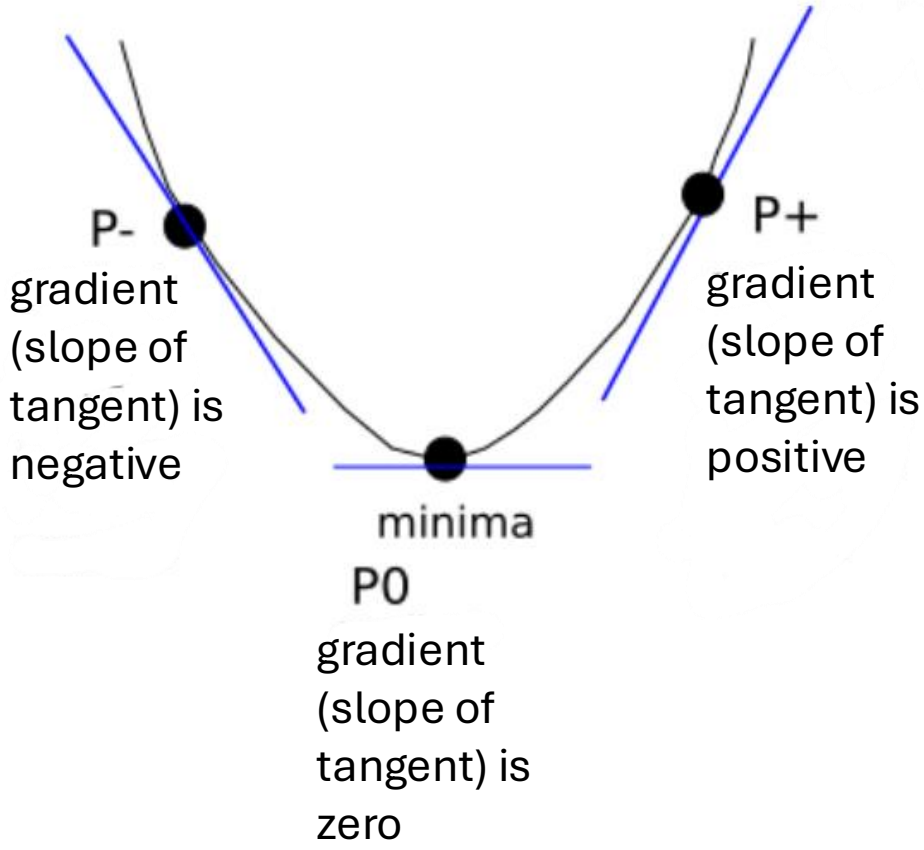
the derivative of a function *f(x)* at a point *a* gives the slope of the **tangent line** to the graph of the function at that point.



- A **positive derivative** means the function is increasing.
- A **negative derivative** means the function is decreasing.

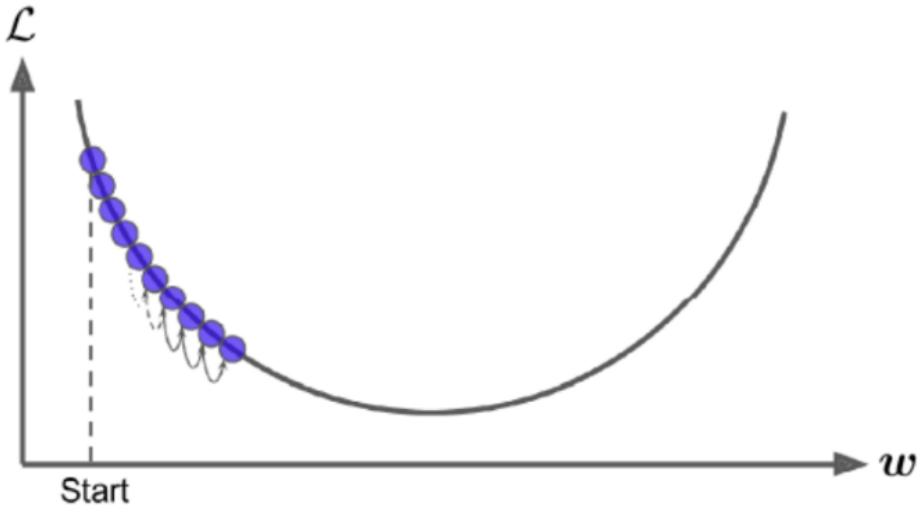
$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w})$$

- If $\nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w})$ is positive means the function is increasing, so by subtraction means that we go in the opposite direction (of decreasing the function)
- If $\nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w})$ is negative means the function is decreasing, so by subtraction means that we keep the direction of decreasing

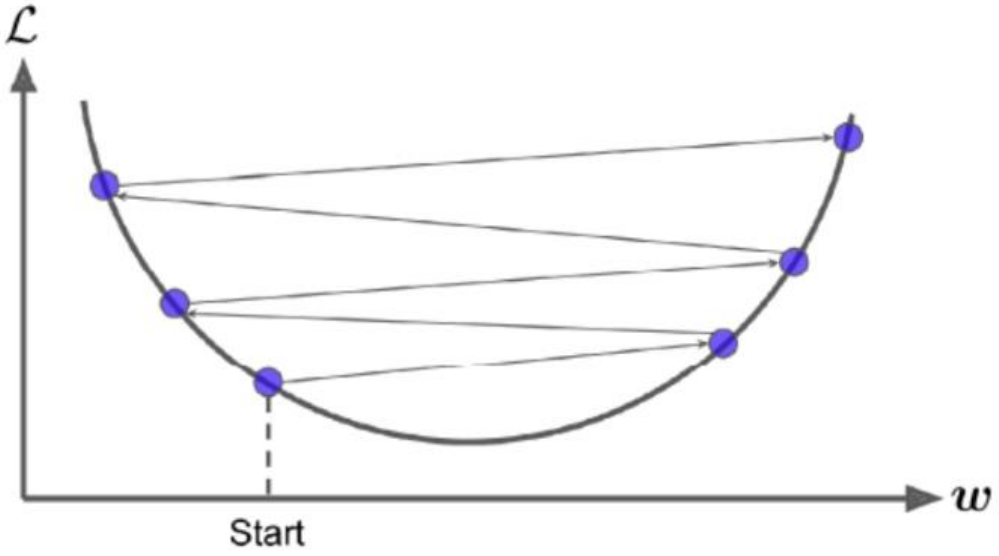


Some notes...

If the learning rate is **too small**, the algorithm will have to go through many iterations to converge, which will take a long time

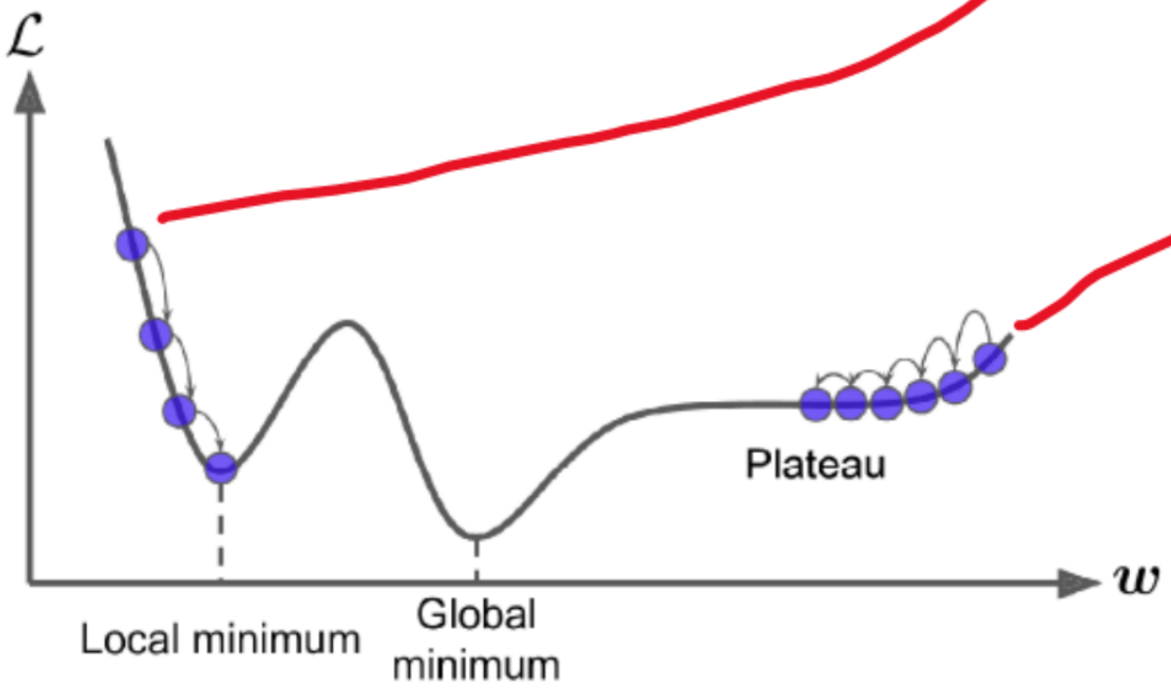


If the learning rate is **too high**, we might jump across the valley and end up on the other side, possibly even higher up than we were before and this might make the algorithm diverge



Some notes...

- Not all functions look nice, regular bowls
- There may be holes, ridges, plateaus, etc., making convergence to the minimum difficult



If the random initialization starts the algorithm on the left, then it will converge to a *local minimum*, which is not as good as the *global minimum*

If the random initialization starts the algorithm on the right, then it will take a very long time to cross the plateau and if we stop too early, we will never reach the *global minimum*

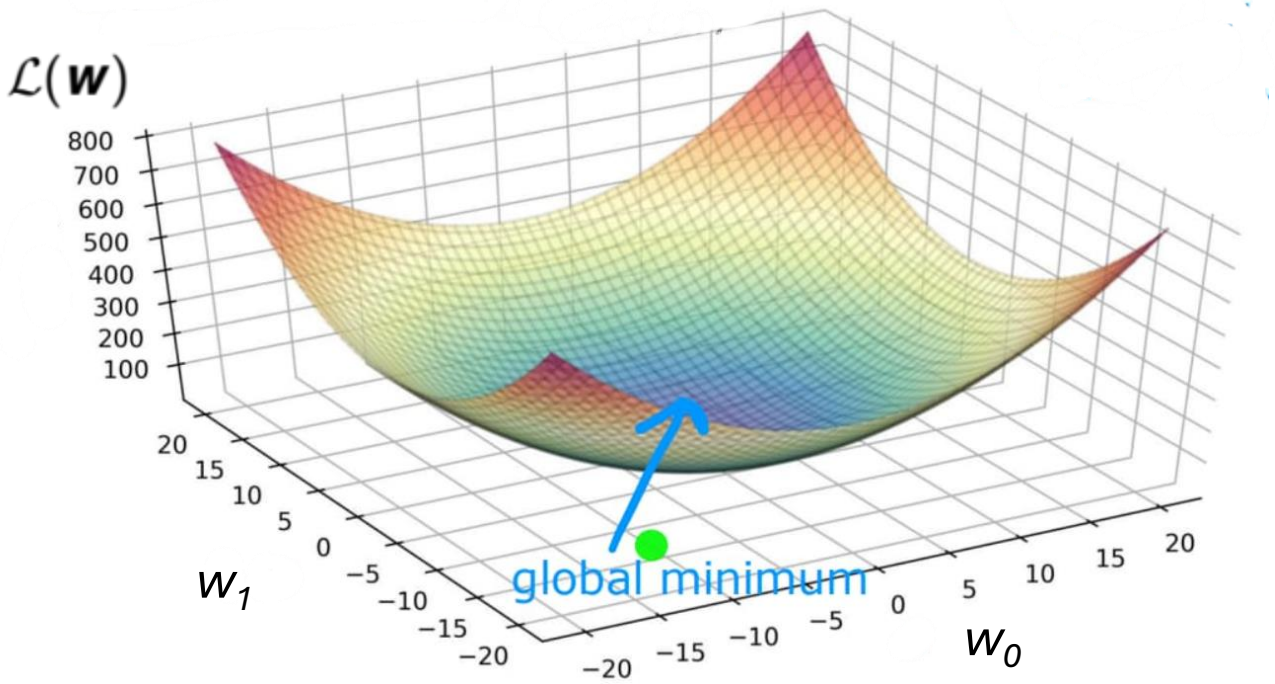


see here 6.4 Gabor loss function: [UDL Interactive Figures]

Some notes...

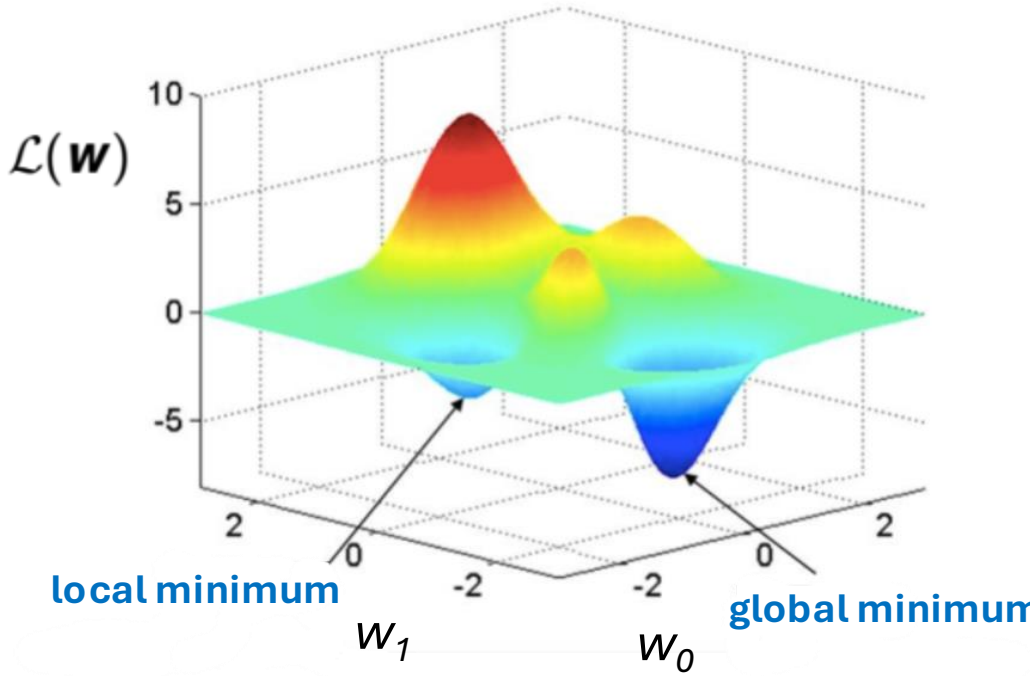
- Fortunately, the mean squared error loss function for linear regression is always a convex function ("ține apa")
- This implies that there are no local minima, just one global minimum

Mean squared error loss function



vs

Other loss function



Code part

Method 1. Implement manually

- **batch gradient descent** → uses the whole batch of training data at every step;

- batch gradient descent:

```
eta = 0.2  # learning rate
n_epochs = 1000
n = 100

w = np.random.randn(2,1)  # random initialization

for iteration in range(n_epochs):
    gradients = 1/n * X_1.T.dot(X_1.dot(w) - y)
    w = w - eta * gradients

w
```

```
array([[4.21509616],
       [2.77011339]])
```

- $\nabla_w \mathcal{L}(w) = \frac{1}{n} X^T (Xw - y)$

go back to slide 24 to believe ☺

Method 1. Implement manually

- **stochastic gradient descent** → picks a random instance in the training set at every step and computes the gradients based only on that single instance;
- stochastic gradient descent:

```
eta = 0.2
n_epochs = 1000
n = 100 # size of the dataset

w = np.random.randn(2,1)  # random initialization

for epoch in range(n_epochs):
    for i in range(n):
        random_index = np.random.randint(n)
        xi = X_1[random_index:random_index+1]
        yi = y[random_index:random_index+1]
        gradients = xi.T.dot(xi.dot(w) - yi)
        w = w - eta * gradients

w
```

```
array([[4.17686315],
       [2.85986577]])
```

Method 1. Implement manually

- **mini-batch gradient descent** → computes the gradients on small random sets of instances called *mini-batches*;

- mini-batch gradient descent:

```
eta = 0.2
n_epochs = 1000
batch_size = 20

w = np.random.randn(2,1)  # random initialization

for epoch in range(n_epochs):
    shuffled_indices = np.random.permutation(n)
    X_1_shuffled = X_1[shuffled_indices]
    y_shuffled = y[shuffled_indices]
    for i in range(0, n, batch_size):
        xi = X_1_shuffled[i:i+batch_size]
        yi = y_shuffled[i:i+batch_size]
        gradients = 1/batch_size * xi.T.dot(xi.dot(w) - yi)
        w = w - eta * gradients

w
```

```
array([[4.19811521],
       [2.75356112]])
```

Method 2. Use scikit-learn

- *SGDRegressor* class → linear regression using stochastic gradient descent:

```
from sklearn.linear_model import SGDRegressor

sgd_reg = SGDRegressor(max_iter=1000, tol=1e-3, penalty=None, eta0=0.2, learning_rate='constant', random_state=42)
sgd_reg.fit(X, y.ravel())

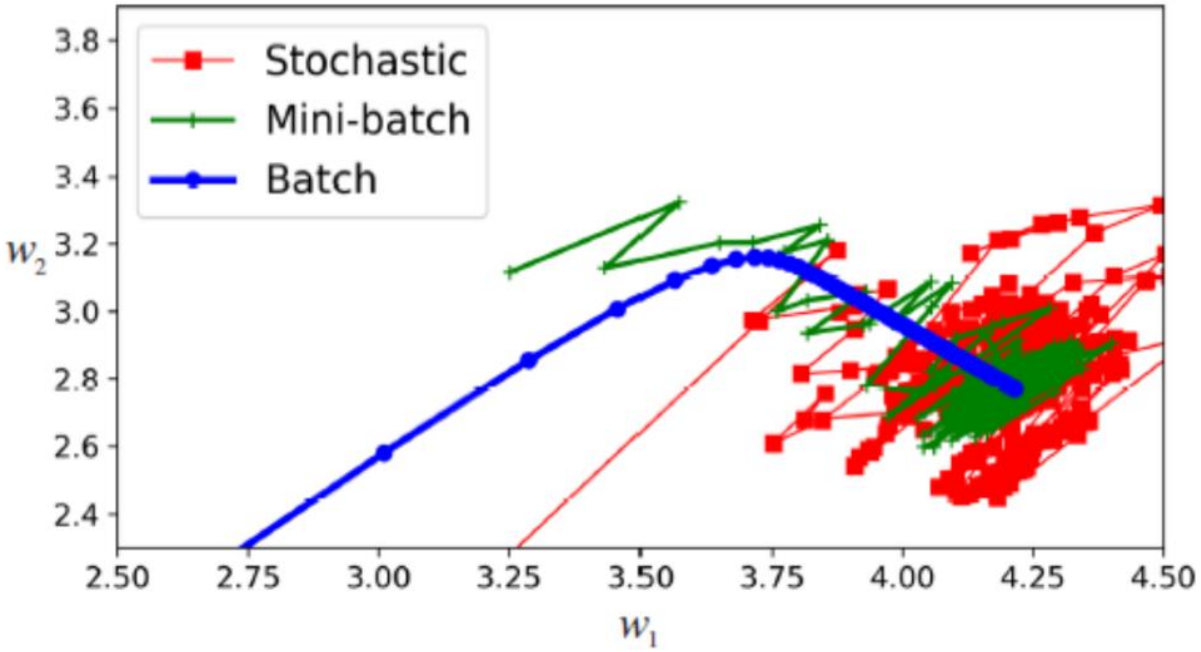
sgd_reg.intercept_, sgd_reg.coef_
```

```
(array([3.99858866]), array([2.60693164]))
```

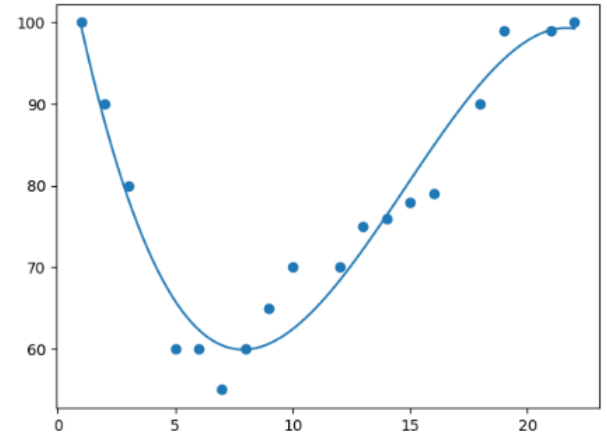
Some notes...

When to Use Each:

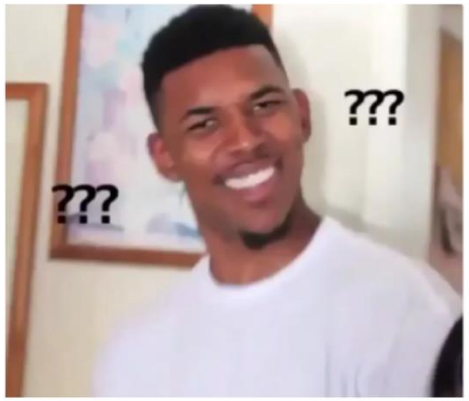
- **Batch Gradient Descent**: Works best when you have smaller datasets that can fit into memory, and you need stable and deterministic updates.
- **Stochastic Gradient Descent**: Ideal for large datasets or online learning when you need to update the model quickly and efficiently.
- **Mini-Batch Gradient Descent**: Generally the most commonly used variant, as it combines the benefits of both BGD and SGD. It is well-suited for larger datasets where you can take advantage of parallelization and hardware acceleration (like GPUs).



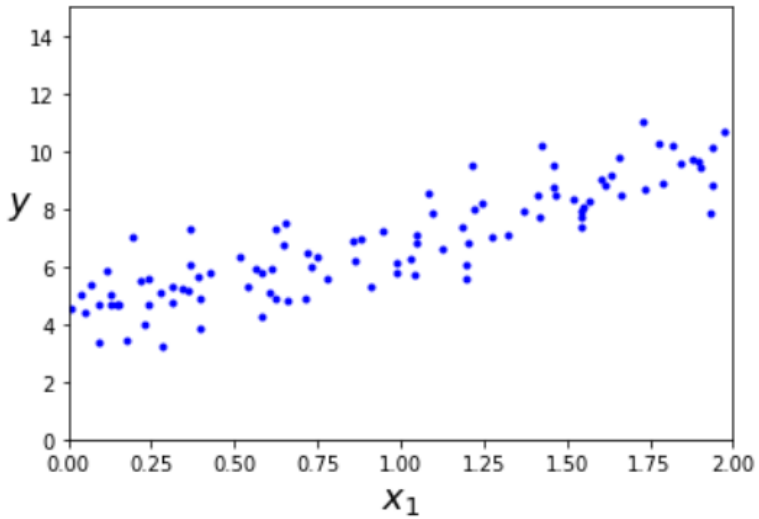
3) Polynomial regression



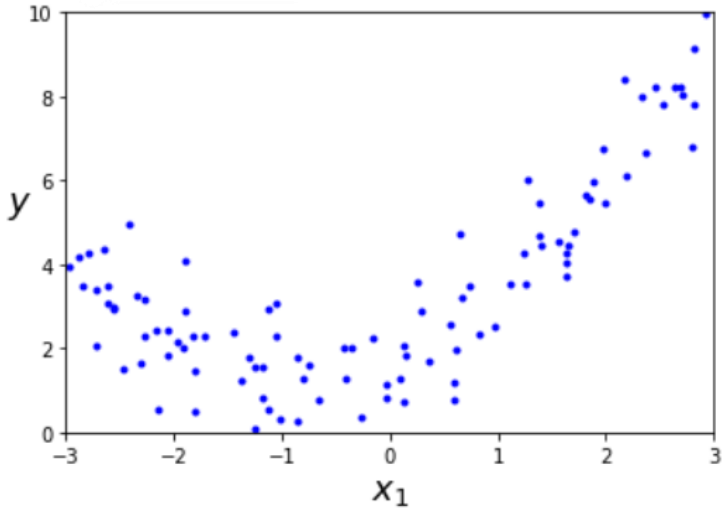
- **polynomial regression** $\rightarrow$ add powers of each feature as new features, then train a linear model on this extended set of features;



- what if our data is more complex than a straight line?







➢ surprisingly, we can use a linear model to fit *nonlinear data* !

Example: suppose we have the famous **house price prediction** problem!

We have more features x_1, x_2, ... x_d representing:

- x_1 – number of bedrooms
- x_2 – square footage ("numărul de metri pătrați")
- x_3 – age of the house
- ....

Based on these features, we want to predict the price of the house !

If the data is **linear**, we can apply linear regression:

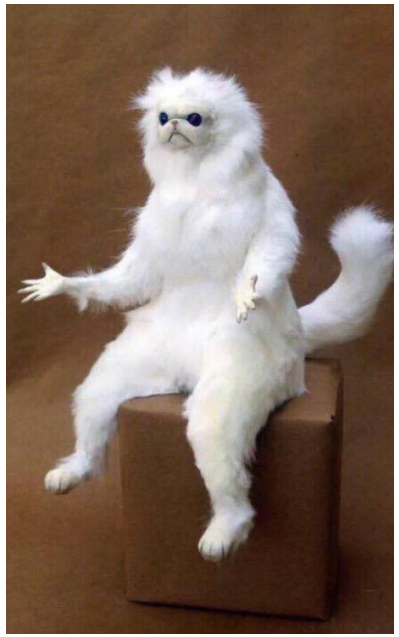
$$\hat{y} = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_d x_d$$

If the data is **non-linear**, we can apply polynomial regression:

$$\hat{y} = w_0 + w_1 x_1 + w_2 x_2 + w_3 x_3 + w_4 x_1^2 + w_5 x_2^2 + w_6 x_3^2 + w_7 x_1 x_2 + w_8 x_1 x_3 + w_9 x_2 x_3 + \cdots$$

NOTE 1): In **polynomial regression**, we can introduce polynomial terms for each feature. This means not only do we include the original features, but we also add higher-order terms (e.g., squares, cubes) and interaction terms between features.

NOTE 2): Polynomial regression is **linear in terms of the parameters**, even though it is **non-linear with respect to the features** !



• The term "**linear**" here refers to how the model parameters (w_0, w_1, w_2,…) appear in the model. They are simply added together and are not multiplied by each other, nor raised to any power.

• On the other hand, the features (x_1, x_2, x_3,…) are raised to higher powers (e.g., x_1^2, x_2^3) and/or multiplied together to create interaction terms (e.g., $x_1 \cdot x_2$)

• This means that polynomial regression is **linear** in the **parameters** but **non-linear** in the **features**.

Example

Consider a simple polynomial regression with a feature x and a quadratic term:

$$\hat{y} = w_0 + w_1 x + w_2 x^2$$

- The model is linear in the parameters (w_0, w_1, w_2) since they are summed linearly.
- The model is non-linear in the feature x because of the squared term (x^2).

Linear vs Non-linear



Linear

- A **linear relationship** means that the change in the output (y) is directly proportional to the change in the input (x).
- In other words, if you plot y versus x, the points roughly form a straight line.
- Example: The equation $y = 3x + 2$ is linear because it forms a straight line on a graph. For each unit increase in x, y changes by a constant amount (3).

Non-linear

- A **non-linear relationship** means that the change in y is not directly proportional to the change in x, resulting in a curve rather than a straight line.
- The rate of change in y with respect to x varies depending on the value of x.
- Example: The equation $y = 2x^2 + 3$ is non-linear because the relationship between y and x forms a parabolic curve. The effect of x on y changes as x changes.

- So, polynomial regression is defined by:

$$\hat{y} = \boldsymbol{w}^\top \phi(\boldsymbol{x})$$

non-linear transformation of inputs ***x***

- The loss function used for polynomial regression is called called ***root mean squared error (RMSE)***:

$$\sqrt{\frac{1}{n}\sum_{i=1}^{n}(\hat{y}^{(i)} - y^{(i)})^2} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(\boldsymbol{w}^\top \phi(\boldsymbol{x}^{(i)}) - y^{(i)})^2}$$

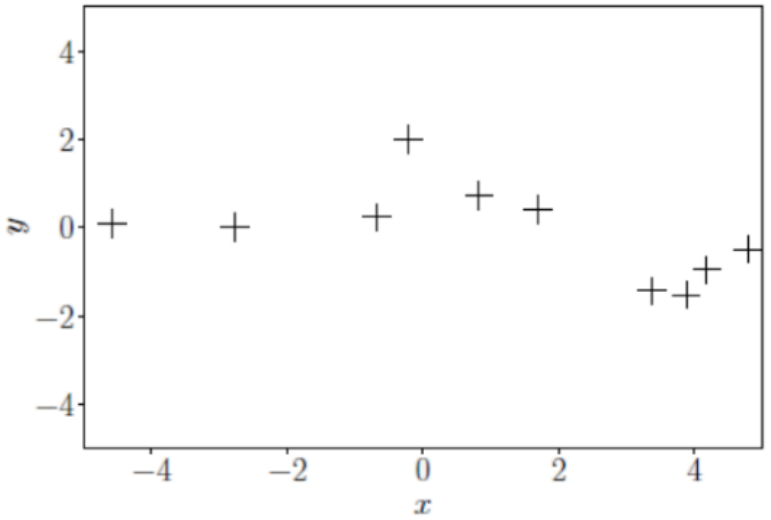
Bias/Variance trade-off



Suppose we have the following data (non-linear):

$$y^{(i)} = \sin\left(\frac{x^{(i)}}{5}\right) + \cos(x^{(i)}) + \epsilon,$$

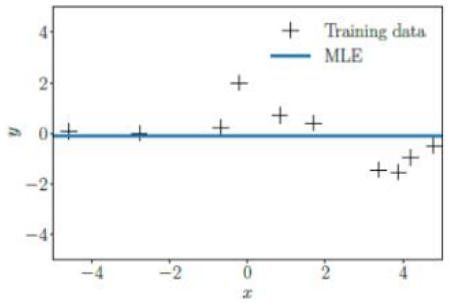
where $\epsilon \sim \mathcal{N}(0, 0.2^2)$



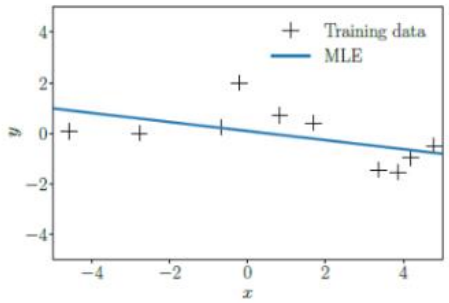
We want to apply polynomial regression for it:

$$\hat{y} = w_0 + w_1 x + w_2 x^2 + w_3 x^3 + \cdots + w_M x^M$$

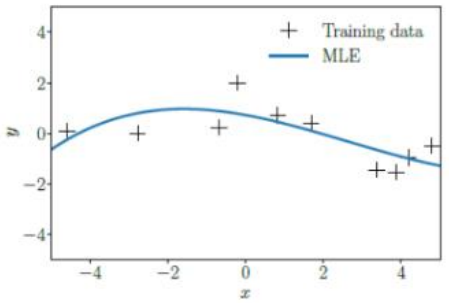
low variance, high bias



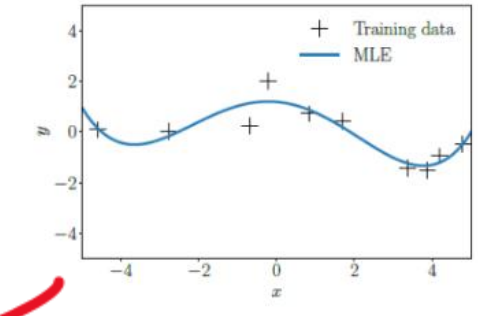
(a) $M = 0$



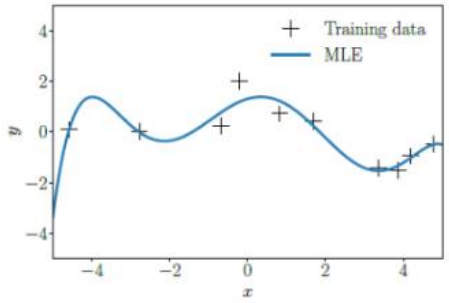
(b) $M = 1$



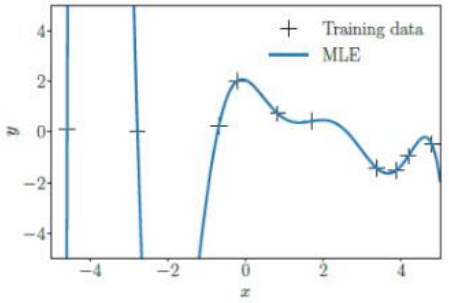
(c) $M = 3$



(d) $M = 4$



(e) $M = 6$



(f) $M = 9$

high variance, low bias

best model !



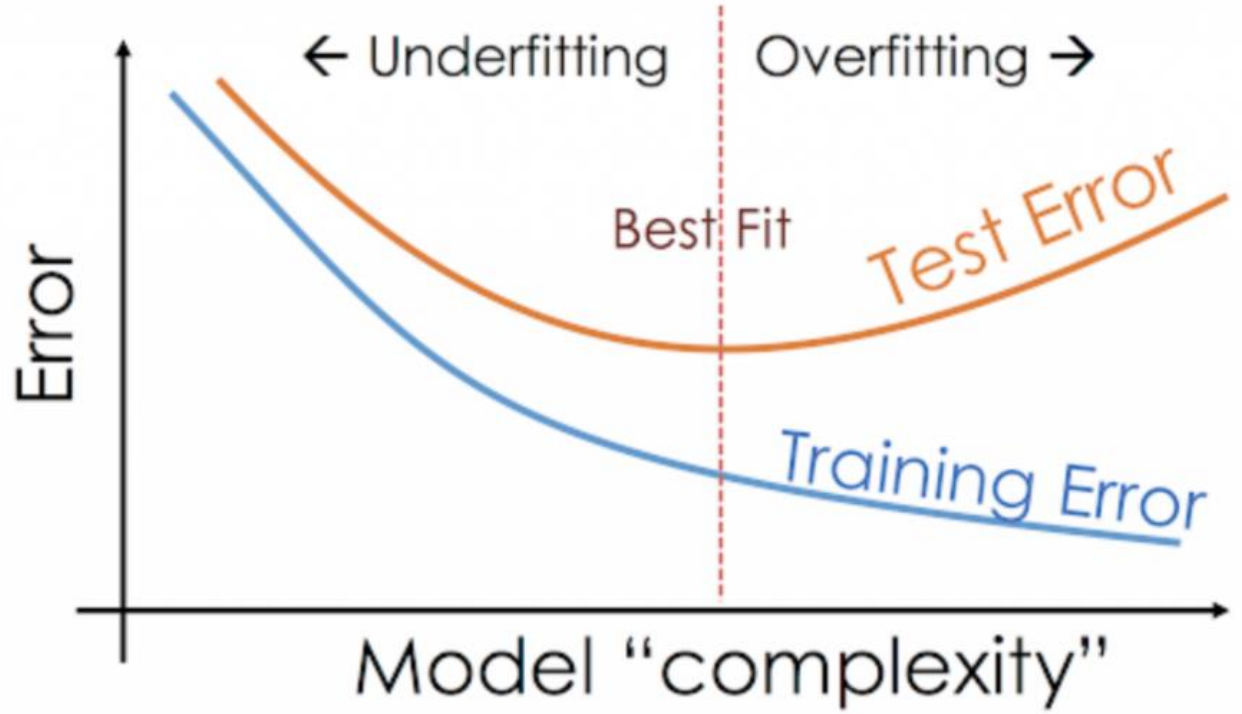
- Increasing a model's complexity (big *M*) → **high variance** and **low bias** → **overfitting**
- Decreasing a model's complexity (small *M*) → **low variance** and **high bias** → **underfitting**

Overfitting/Underfitting



Underfitting occurs when a model is too simple to capture the underlying patterns in the data. It means the model does not fit the training data well and has high **bias** but low **variance**, resulting in poor performance on both training and test datasets.

Overfitting happens when a model is too complex and fits the training data "too well," even capturing noise and random fluctuations. As a result, it has low **bias** but high **variance**—it performs very well on the training data but poorly on unseen data because it lacks generalization



Overfitting/Underfitting

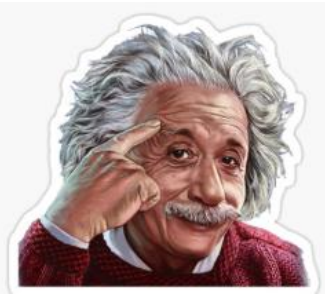




Overfitting (the nerd/egghead 🤓)
- Studies *every single detail* of the textbook, even the footnotes and rare exceptions.
- On the exam, if the exact same question from the book appears, he nails it.
- But if the question is slightly different, he gets confused because he memorized instead of understanding.

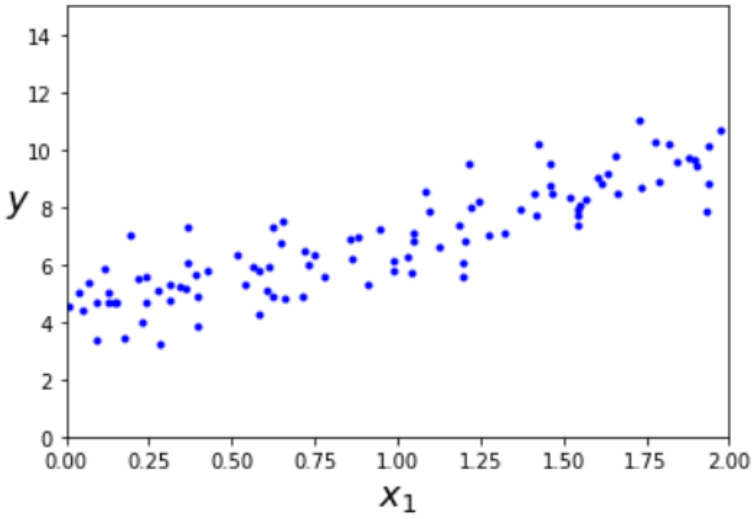


Underfitting (the too-chill guy 😎)
- Barely studies, just skims the summary.
- On the exam, he can only answer the most obvious questions.
- He misses the deeper or trickier ones because he didn't learn enough detail.



Best fit (the actually smart guy 🧠)
- Understands the material well, without obsessing over useless details.
- On the exam, he can handle both the standard and the slightly different questions.

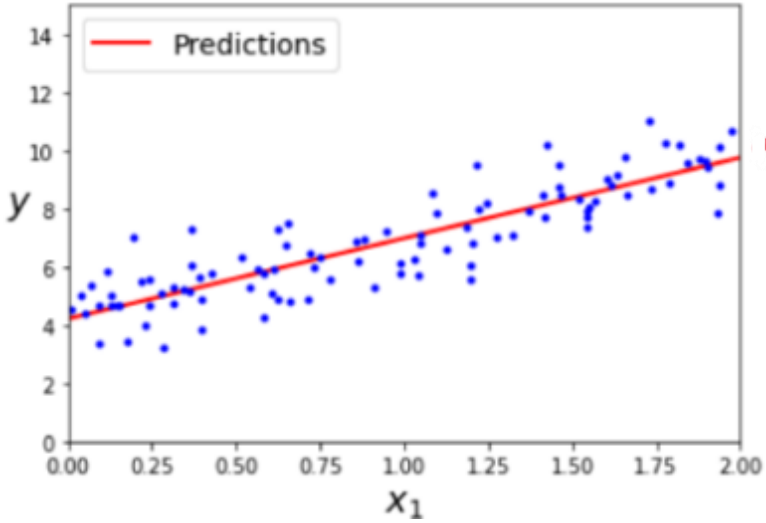
TL; DR

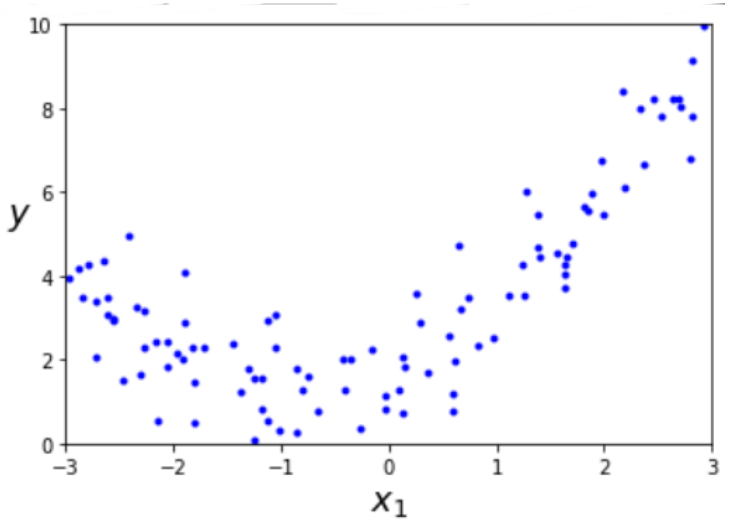


$$\hat{y} = Xw$$

Linear Regression

- Uses a straight line
- Works well if the data has a linear trend
- Struggles with curved/nonlinear patterns

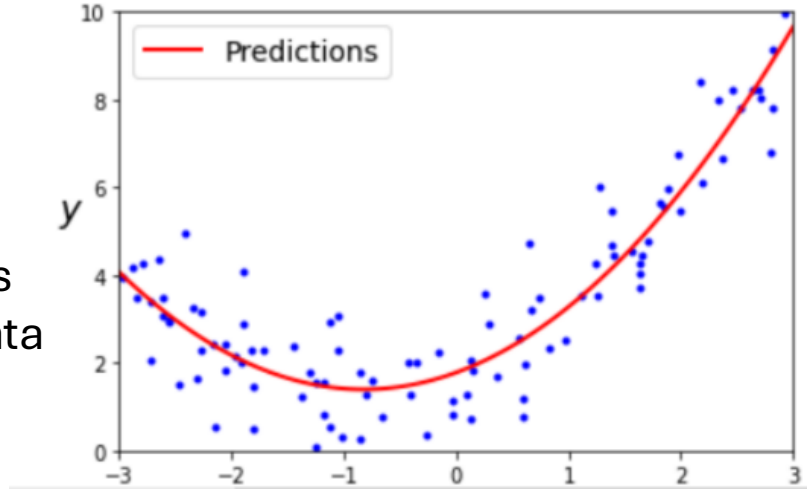




$$\hat{y} = \boldsymbol{w}^\top \phi(\boldsymbol{x})$$

Polynomial Regression

- Transforms inputs into polynomial features
- Can capture curved relationships in the data
- Fits nonlinear trends much better

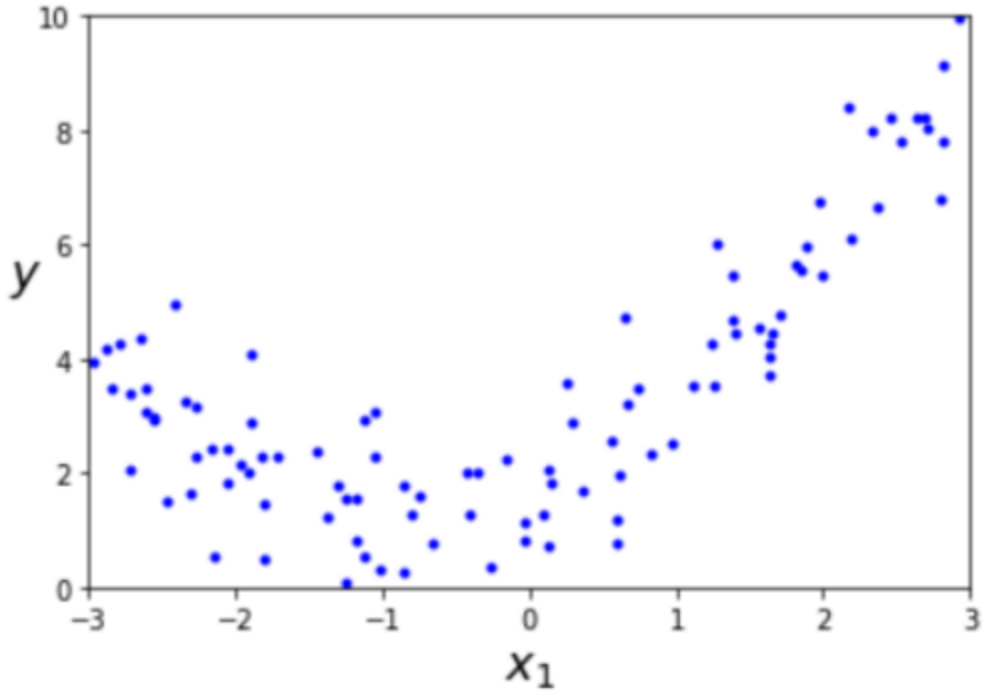


Code part

```python
from sklearn.preprocessing import PolynomialFeatures

n = 100
X = 6 * np.random.rand(n, 1) - 3
y = 0.5 * X**2 + X + 2 + np.random.randn(n, 1)
```

training data




```python
poly_features = PolynomialFeatures(degree=2, include_bias=False)
X_poly = poly_features.fit_transform(X)
```

```
X[0]
array([-0.75275929])

X_poly[0]
array([-0.75275929,  0.56664654])
```

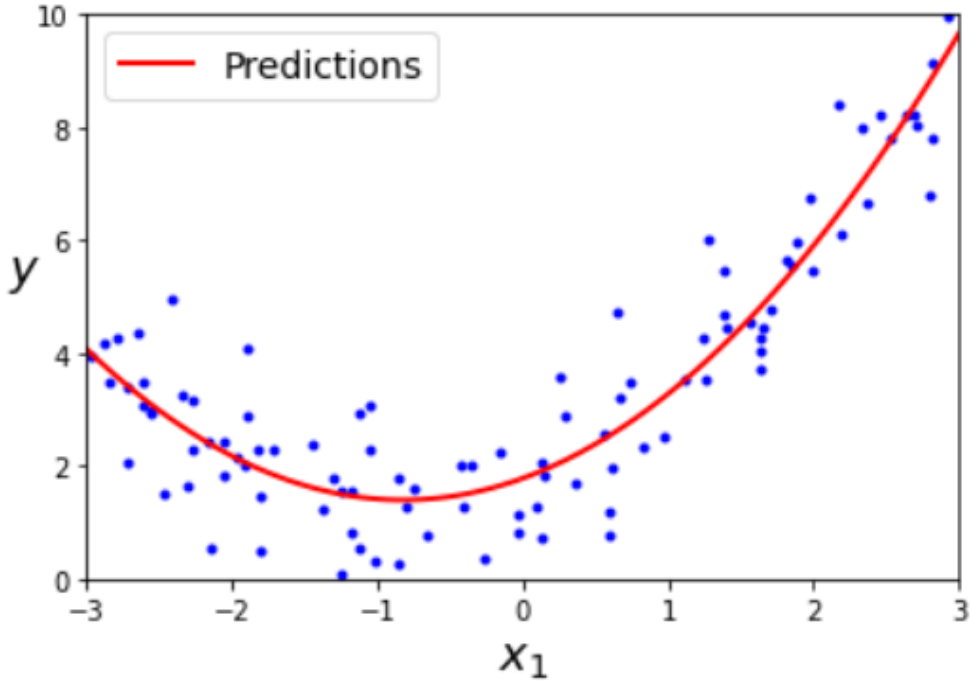
$X[0], X[0]^2$

We use fit_transform() method and PolynomialFeatures class to transform our training data, adding the square (second-degree polynomial) of each feature in the training set as a new feature

Now we can fit a LinearRegression model to this extended training data, thus obtaining polynomial regression:

```
lin_reg = LinearRegression()
lin_reg.fit(X_poly, y)
lin_reg.intercept_, lin_reg.coef_
```

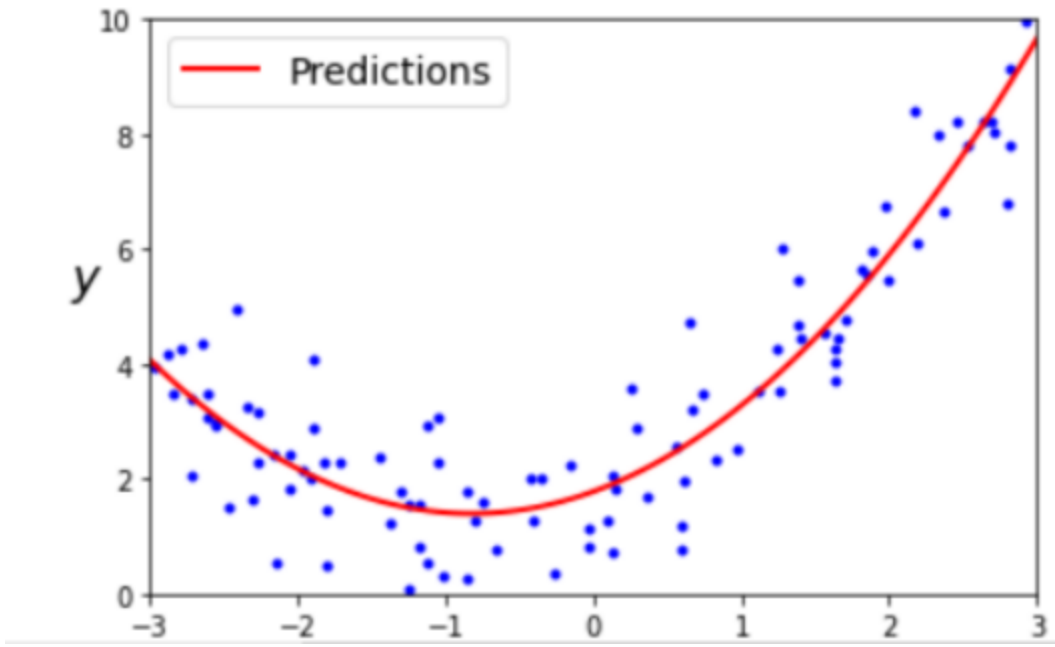
```
(array([1.78134581]), array([[0.93366893, 0.56456263]]))
```



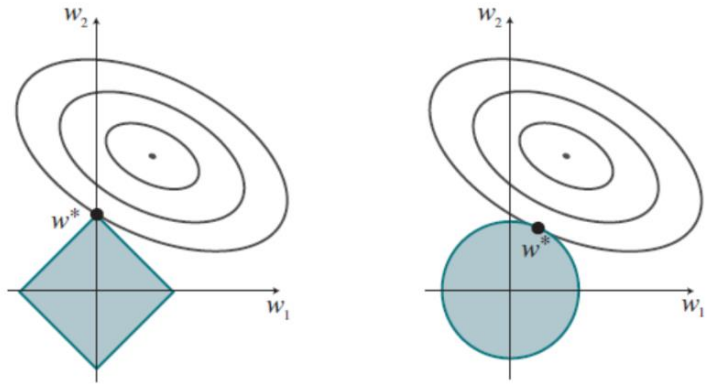
Not bad: the model estimates $y = 0.56x_1^2 + 0.93x_1 + 1.78$, when in fact the original function was $y = 0.5x_1^2 + 1.0x_1 + 2.0 + \text{Gaussian noise}$.

- use the trained polynomial regression model to predict new values across the input range and plot the smooth fitted curve over the data:

```
X_new = np.linspace(-3, 3, 100).reshape(100, 1)
X_new_poly = poly_features.transform(X_new)
y_new = lin_reg.predict(X_new_poly)
plt.plot(X, y, "b.")
plt.plot(X_new, y_new, "r-", linewidth=2, label="Predictions")
plt.xlabel("$x_1$", fontsize=18)
plt.ylabel("$y$", rotation=0, fontsize=18)
plt.legend(loc="upper left", fontsize=14)
plt.axis([-3, 3, 0, 10])
plt.show()
```



4) Regularized linear models



- **Ridge, Lasso,** and **Elastic Net Regression** are 3 different types of linear regression techniques that use **regularization** to prevent overfitting **by adding penalties to the to the loss function**.

- it's almost preferable to have at least *a little bit of regularization*, so generally we should avoid plain linear regression

- **KIND REMINDER**: the loss function for linear regression is:

$$\mathcal{L}(\boldsymbol{w}) = \frac{1}{2n}(\boldsymbol{X}\boldsymbol{w} - \boldsymbol{y})^{\top}(\boldsymbol{X}\boldsymbol{w} - \boldsymbol{y})$$ (see slide 24 ☺)

➢ ridge regression:

$$\mathcal{L}(w) = \frac{1}{2n}(Xw - y)^T(Xw - y) + \frac{1}{2}\lambda\|w\|_2^2,$$

where $\|w\|_2 = \left(\sum_{j=0}^{d} |w_j|^2\right)^{1/2}$ is the ℓ_2 norm of w.

For a vector $\mathbf{w} = [w_1, w_2, \ldots, w_n]$, the L2 norm is:

$$||\mathbf{w}||_2 = \sqrt{w_1^2 + w_2^2 + \cdots + w_n^2}$$

Example

Consider a vector $\mathbf{w} = [3, -4, 1]$:

$$||\mathbf{w}||_2 = \sqrt{3^2 + (-4)^2 + 1^2} = \sqrt{9 + 16 + 1} = \sqrt{26} \approx 5.1$$

➢ lasso regression:

$$\mathcal{L}(w) = \frac{1}{2n}(Xw - y)^T(Xw - y) + \lambda\|w\|_1,$$

where $\|w\|_1 = \sum_{j=0}^{d} |w_j|$ is the ℓ_1 norm of w.

For a vector $\mathbf{w} = [w_1, w_2, \ldots, w_n]$, the L1 norm is:

$$||\mathbf{w}||_1 = |w_1| + |w_2| + \cdots + |w_n|$$

Example

Consider a vector $\mathbf{w} = [3, -4, 1]$:

$$||\mathbf{w}||_1 = |3| + |-4| + |1| = 3 + 4 + 1 = 8$$

➢ elastic net regression:

$$\mathcal{L}(w) = \frac{1}{2n}(Xw - y)^T(Xw - y) + r\lambda\|w\|_1 + \frac{1 - r}{2}\lambda\|w\|_2^2,$$

where $r \in [0, 1]$.

Code part

ridge regression:

```
from sklearn.linear_model import Ridge

n = 20
X = 3 * np.random.rand(n, 1)
y = 1 + 0.5 * X + np.random.randn(n, 1) / 1.5
X_new = np.linspace(0, 3, 100).reshape(100, 1)

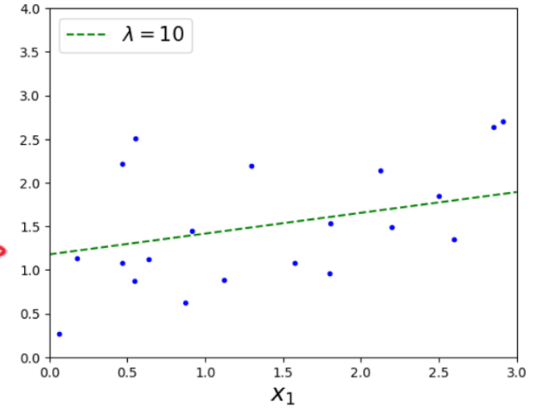
ridge_reg = Ridge(alpha=10)
ridge_reg.fit(X, y)
y_new_reg = ridge_reg.predict(X_new)
```

```
ridge_reg.intercept_, ridge_reg.coef_

(array([1.00650911]), array([[0.36280369]]))
```

- $\lambda = \text{alpha} = 10$

$\hat{y} = \mathbf{1.00} + \mathbf{0.36} \cdot x_1$



lasso regression:

```
from sklearn.linear_model import Lasso

n = 20
X = 3 * np.random.rand(n, 1)
y = 1 + 0.5 * X + np.random.randn(n, 1) / 1.5
X_new = np.linspace(0, 3, 100).reshape(100, 1)

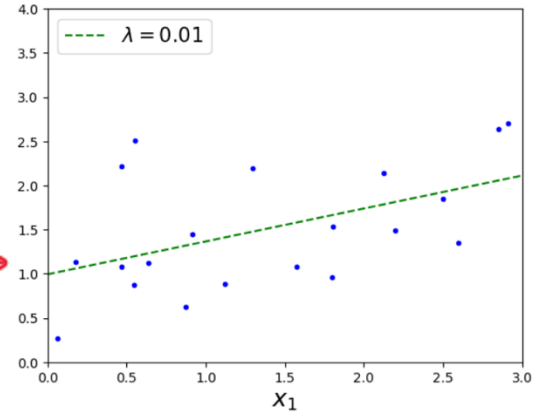
lasso_reg = Lasso(alpha=0.01)
lasso_reg.fit(X, y)
y_new_reg = lasso_reg.predict(X_new)
```

```
lasso_reg.intercept_, lasso_reg.coef_

(array([0.99270036]), array([0.37286027]))
```

- $\lambda = \text{alpha} = 0.01$

$\hat{y} = \mathbf{0.99} + \mathbf{0.37} \cdot x_1$



elastic net regression:

```
from sklearn.linear_model import ElasticNet

n = 20
X = 3 * np.random.rand(n, 1)
y = 1 + 0.5 * X + np.random.randn(n, 1) / 1.5
X_new = np.linspace(0, 3, 100).reshape(100, 1)

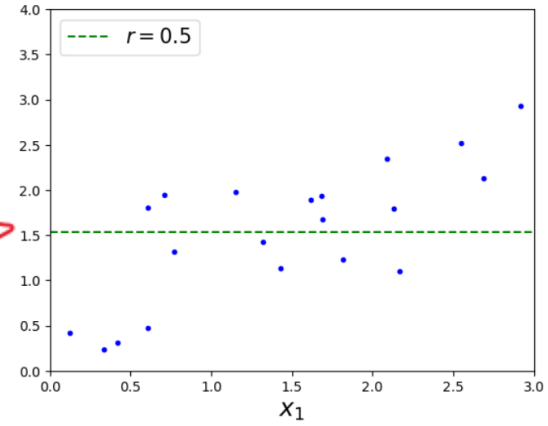
elastic_net = ElasticNet(alpha=1, l1_ratio=0.5, random_state=42)
elastic_net.fit(X, y)
y_new_reg =elastic_net.predict(X_new)
```

```
elastic_net.intercept_, elastic_net.coef_

(array([1.50467735]), array([0.]))
```

- $r = \text{l1_ratio} = 0.5$

$\hat{y} = \mathbf{1.50} + \mathbf{0.00} \cdot x_1$



…or alternatively using stochastic gradient descent…

- ridge regression:

```
sgd_reg = SGDRegressor(penalty="l2", alpha=1, max_iter=1000, tol=1e-3, random_state=42)
sgd_reg.fit(X, y.ravel())
sgd_reg.intercept_, sgd_reg.coef_
```

```
(array([0.88105339]), array([0.33523244]))
```

- lasso regression:

```
sgd_reg = SGDRegressor(penalty="l1", alpha=1, max_iter=1000, tol=1e-3, random_state=42)
sgd_reg.fit(X, y.ravel())
sgd_reg.intercept_, sgd_reg.coef_
```

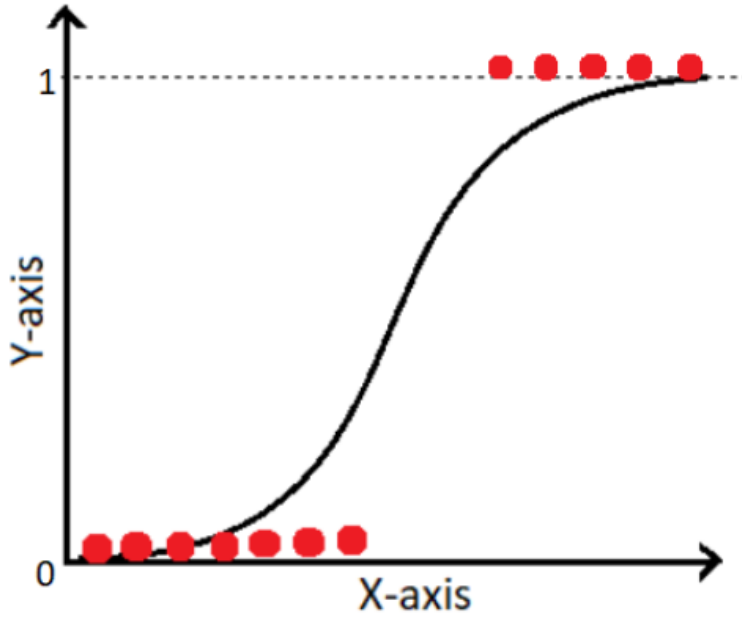
```
(array([1.017502]), array([0.07016255]))
```

- elastic net regression:

```
sgd_reg = SGDRegressor(penalty="elasticnet", alpha=1, l1_ratio=0.5, max_iter=1000, tol=1e-3, random_state=42)
sgd_reg.fit(X, y.ravel())
sgd_reg.intercept_, sgd_reg.coef_
```

```
(array([0.98691798]), array([0.20034481]))
```

5) Logistic regression



A simple example to understand the problem

Problem to Solve

• We want to predict **whether a student will pass or fail** an exam.

• Input: **hours studied**.

• Output: **binary result** (Pass = 1, Fail = 0).

Student Name	Hours Studied	Result (Pass=1, Fail=0)
Alice	0.5	0
Bob	0.75	0
Charlie	1	0
David	1.25	0
Emma	1.5	0
Frank	1.75	0
George	1.75	1
Hannah	2	0
Ian	2.25	1
Jane	2.5	0
Kevin	2.75	1
Liam	3	0
Marie	3.25	1
Nora	3.5	0
Oliver	4	1
Paul	4.25	1
Rachel	4.5	1
Sam	4.75	1
Tom	5	1
Victor	5.5	1

How Logistic Regression Works (Intuition)

• We start **just like linear regression**:

– Multiply inputs by weights and add them up (a linear combination):

$$w^T x$$

• Instead of keeping that raw number, we **pass it through a special function** → squashes any value into a range between **0 and 1**. The output is interpreted as a **probability** (e.g. *70% chance to pass the exam*):

– Small values → close to 0 (low probability).

– Large values → close to 1 (high probability).

$$\hat{p} = \sigma(w^T x)$$

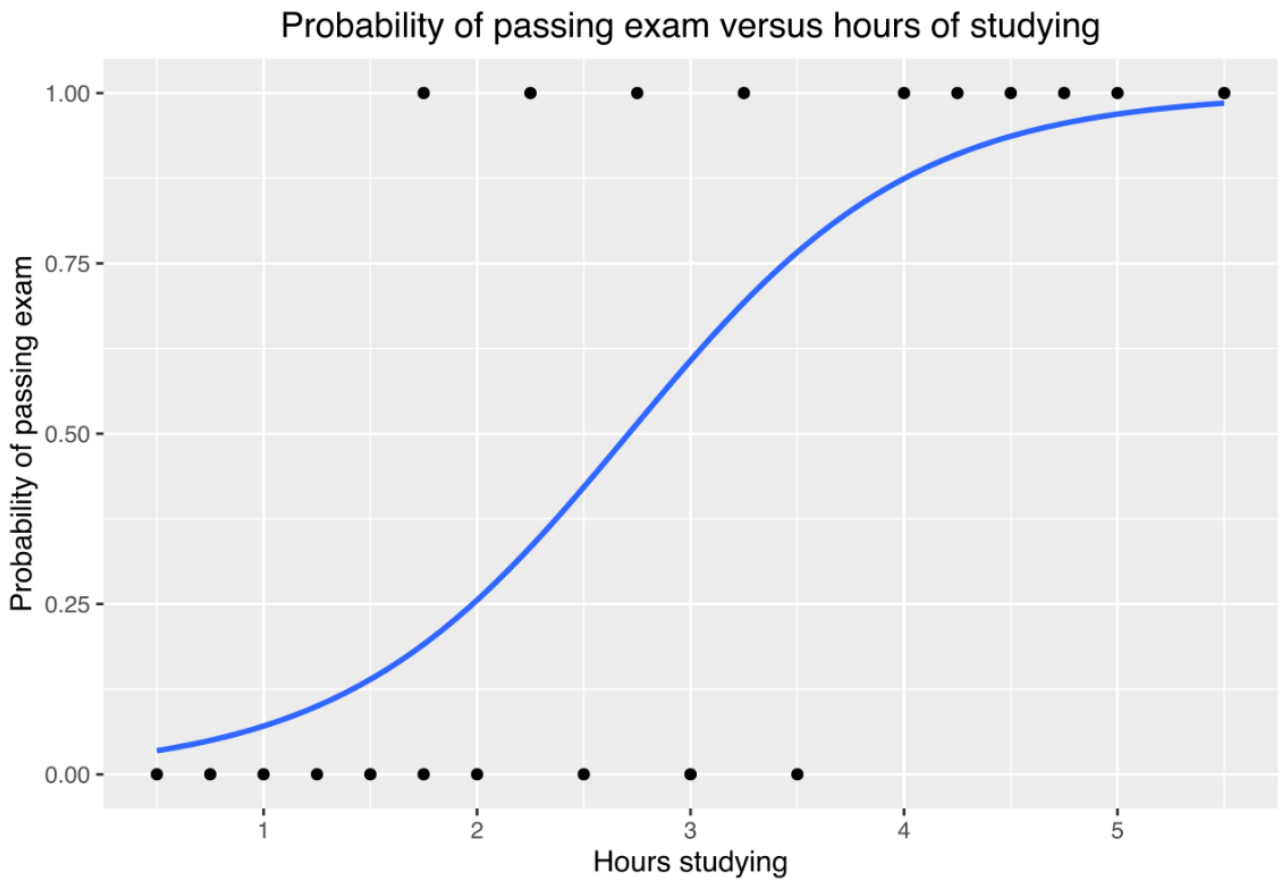
• Finally, we apply a **threshold** (commonly 0.5):

– ≥ 0.5 → predict **Pass**

– < 0.5 → predict **Fail**

$$\hat{y} = \begin{cases} 0, & \text{if } \hat{p} < 0.5 \\ 1, & \text{if } \hat{p} \geq 0.5 \end{cases}$$

Student Name	Hours Studied	Result (Pass=1, Fail=0)
Alice	0.5	0
Bob	0.75	0
Charlie	1	0
David	1.25	0
Emma	1.5	0
Frank	1.75	0
George	1.75	1
Hannah	2	0
Ian	2.25	1
Jane	2.5	0
Kevin	2.75	1
Liam	3	0
Marie	3.25	1
Nora	3.5	0
Oliver	4	1
Paul	4.25	1
Rachel	4.5	1
Sam	4.75	1
Tom	5	1
Victor	5.5	1



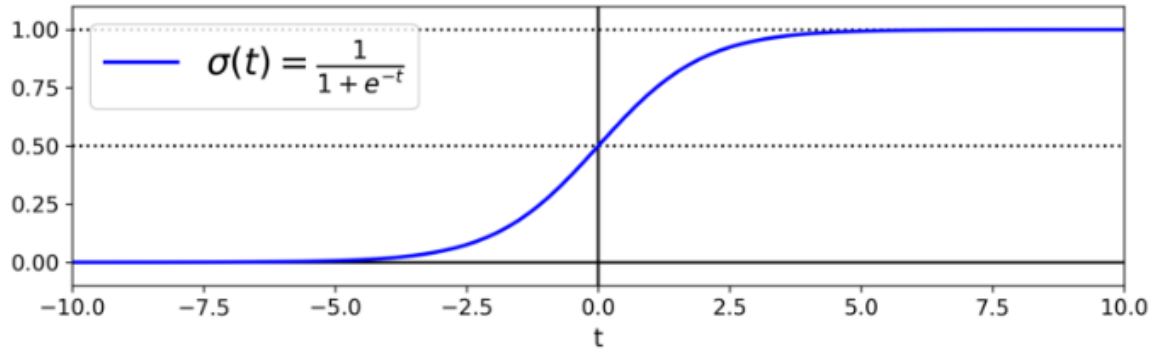
- **logistic regression** is commonly used to estimate the probability that an instance belongs to a particular class ($q = 2$ classes); → **it's a binary classifier !**

- the idea of this method is to compute the probabilities of an input belonging to each class and then predict the class with the highest probability;

- a logistic regression model computes a weighted sum of the input features (plus a bias term) and outputs:

$$\hat{p} = \sigma(w^T x),$$

where $\underbrace{\sigma(t) = \frac{1}{1 + exp(-t)}, \sigma(t) \in [0, 1]}_{\text{sigmoid function}}$;



- once the logistic regression model has estimated the probability $\hat{p}$ that an instance $\boldsymbol{x}$ belongs to the positive class, it can make its prediction $\hat{y}$ easily as:

$$\hat{y} = \begin{cases} 0, & \text{if } \hat{p} < 0.5 \\ 1, & \text{if } \hat{p} \geq 0.5 \end{cases} \quad \Longleftrightarrow \quad \hat{y} = \begin{cases} 0, & \text{if } \sigma(\boldsymbol{w}^T \boldsymbol{x}) < 0.5 \\ 1, & \text{if } \sigma(\boldsymbol{w}^T \boldsymbol{x}) \geq 0.5 \end{cases} \quad \Longleftrightarrow \quad \hat{y} = \begin{cases} 0, & \text{if } \boldsymbol{w}^T \boldsymbol{x} < 0 \\ 1, & \text{if } \boldsymbol{w}^T \boldsymbol{x} \geq 0 \end{cases}$$

- The loss function for a single training instance x is:

$$I(\boldsymbol{w}) = \begin{cases} -\log \hat{p}, & \text{if } y = 1 \\ -\log(1 - \hat{p}), & \text{if } y = 0 \end{cases} \quad \Longleftrightarrow \quad I(\boldsymbol{w}) = -y \log \hat{p} - (1 - y) \log(1 - \hat{p})$$

- this loss function makes sense because $-\log t$ grows very large when t approaches 0, so the loss will be large if the model estimates a probability close to 0 for a positive instance, and it will also be very large if the model estimates a probability close to 1 for a negative instance
- on the other hand, $-\log t$ is close to 0 when t is close to 1, so the loss will be close to 0 if the estimated probability is close to 0 for a negative instance or close to 1 for a positive instance, which is precisely what we want

- the loss function over the whole training set is the average loss over all training instances:

$$\mathcal{L}(\boldsymbol{w}) = -\frac{1}{n}\sum_{i=1}^{n}\left[y^{(i)}\log\hat{p}^{(i)} + (1-y^{(i)})\log(1-\hat{p}^{(i)})\right]$$

→ log loss

- the bad news is that there is no known closed-form equation to compute the value of $\boldsymbol{w}$ that minimizes this loss function (there is no equivalent of the normal equation)
- the good news is that this loss function is convex, so gradient descent (or any other optimization algorithm) is guaranteed to find the global minimum (if the learning rate is not too large and we wait long enough)

- the gradient of the loss function is given by:

$$\nabla_{\boldsymbol{w}}\mathcal{L}(\boldsymbol{w}) = \frac{1}{n}\boldsymbol{X}^{\top}(\sigma(\boldsymbol{X}\boldsymbol{w}) - \boldsymbol{y})$$

Code part

- iris dataset (150 examples, 3 classes encoded as $\{0, 1, 2\}$, 4 features);

- a famous dataset that contains the sepal and petal length and width of 150 iris flowers of 3 different species: iris setosa, iris versicolor, and iris virginica.

```
from sklearn import datasets
iris_dataset = datasets.load_iris()
```

```
iris_dataset.keys()
```

```
dict_keys(['data', 'target', 'frame', 'target_names', 'DESCR', 'feature_names', 'filename', 'data_module'])
```

```
iris_dataset['target_names']
```

```
array(['setosa', 'versicolor', 'virginica'], dtype='<U10')
```

→ 3 classes

```
iris_dataset['feature_names']
```

```
['sepal length (cm)',
 'sepal width (cm)',
 'petal length (cm)',
 'petal width (cm)']
```

→ 4 features

Inspecting the data...

```
iris_dataset['data'].shape
```

```
(150, 4)
```

```
iris_dataset['data'][:5]
```

```
array([[5.1, 3.5, 1.4, 0.2],
       [4.9, 3. , 1.4, 0.2],
       [4.7, 3.2, 1.3, 0.2],
       [4.6, 3.1, 1.5, 0.2],
       [5. , 3.6, 1.4, 0.2]])
```

first flower from the dataset

second flower from the dataset

```
iris_dataset['target'].shape
```

```
(150,)
```

```
iris_dataset['target']
```

```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2])
```

```
iris_dataset['target_names']
```

```
array(['setosa', 'versicolor', 'virginica'], dtype='<U10')
```

0: setosa
1: versicolor
2: virginica

split the dataset as: 75% for training set and 25% for test set;

```
X_train, X_test, y_train, y_test = train_test_split(iris_dataset['data'], iris_dataset['target'], random_state=0)
```



- Before making the split, the train_test_split() method shuffles the dataset using a pseudorandom number generator.
- If we just took the last 25% of the data as a test set, all the data points would have the label 2, as the data points are sorted by the label (see the output for iris['target'] shown earlier).
- Using a test set containing only one of the three classes would not tell us much about how well our model generalizes, so we shuffle our data to make sure the test data contains data from all classes.

Sanity check:

```
X_train.shape, y_train.shape
```

```
((112, 4), (112,))
```

```
X_test.shape, y_test.shape
```

```
((38, 4), (38,))
```

NOTE: If you want to split the dataset different than 75% and 25% you need to modify the default parameters of the train_test_split() function

- Let's now use logistic regression to build a binary classifier to detect the iris virginica type. Basically, we will have two classes: iris virginica and not iris virginica. We transform the targets correspondingly:

```
y_train_binary = (y_train == 2).astype(int) # 1 if iris virginica, else 0
y_test_binary = (y_test == 2).astype(int) # 1 if iris virginica, else 0
```

- Now let's train a LogisticRegression model:

```
from sklearn.linear_model import LogisticRegression

log_reg = LogisticRegression(random_state=42)
log_reg.fit(X_train, y_train_binary)
```

- In order to evaluate the accuracy of the logistic regression model on the test set, we use the score() method:

```
log_reg.score(X_test, y_test_binary)
```

```
0.9736842105263158
```

→ our model correctly classified 97.3% of the examples in the test set !

- We can use the predict() method to apply the trained model on the first example in the test set:

```
log_reg.predict([X_test[0]])
```

```
array([1])
```

→ The model predicted that the first example in the test set is an iris virginica!

- We can also inspect the probabilities the model assigned to the negative and positive classes, respectively, using the predict_proba() method:

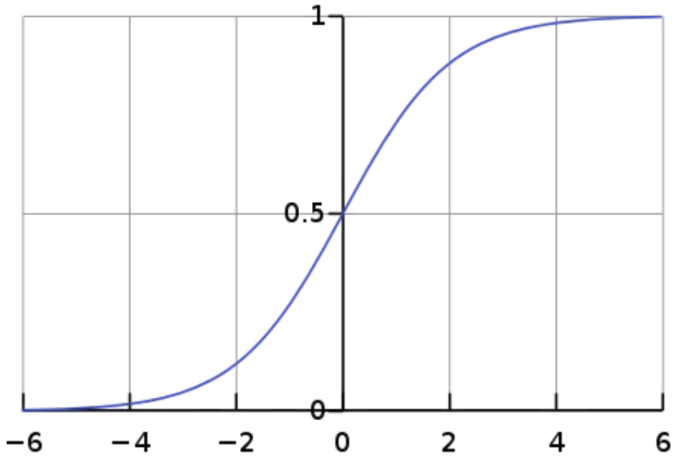
```
log_reg.predict_proba([X_test[0]])
```

```
array([[0.08458525, 0.91541475]])
```



The model is 91.5% sure that the example is an iris virginica!

6) Softmax regression



A simple example to understand the problem

Student Name	Hours Studied	Class participation (0-10)	Final Grade
Alice	5	8	A
Bob	4	7	B
Charlie	2	6	C
David	1	4	D
Emma	3	5	C
Frank	6	9	A
George	2	3	D
Hannah	4	7	B
Ian	5	8	A
Jane	3	6	B

The features are:

- **Hours Studied**: Continuous variable indicating how many hours the student studied.
- **Class Participation**: A score from 0 to 10 indicating how actively the student participated in class.

The target variable (Grade) has four classes: A, B, C, and D.

- **softmax regression** is a generalization of logistic regression to multi-class problems ($q > 2$ classes);

- Softmax regression steps:
  1. **Linear Transformation**: compute a linear combination of input features using class-specific weights for each class;

$$o_k(\mathbf{x}) = \mathbf{w}_k^T \mathbf{x}, \quad k = \overline{1, q}.$$

NOTE:
each class has its own dedicated parameter vector $\boldsymbol{w}_k \in \mathbb{R}^d$

  2. **Softmax Function**: apply this function to the computed linear combinations to convert them into probabilities;

$$\hat{p}_k = softmax(\mathbf{o}(\mathbf{x}))_k = \frac{exp(o_k(\mathbf{x}))}{\sum_{l=1}^{q} exp(o_l(\mathbf{x}))}.$$

vector containing the scores of each class for the instance $\boldsymbol{x}$

the estimated probability that the instance $\boldsymbol{x}$ belongs to class k

  3. **Prediction**: the class with the highest probability is the output class;

$$\hat{y} = \underset{k}{argmax}\, \hat{p}_k.$$

- The loss function used for softmax regression is called called ***cross entropy*** loss and it captures the difference between predicted probabilities and the actual labels:

$$\mathcal{L}(\mathbf{W}) = -\frac{1}{n}\sum_{i=1}^{n}\sum_{k=1}^{q} y_k^{(i)} log\, \hat{p}_k^{(i)}$$

number of training instances

number of classes

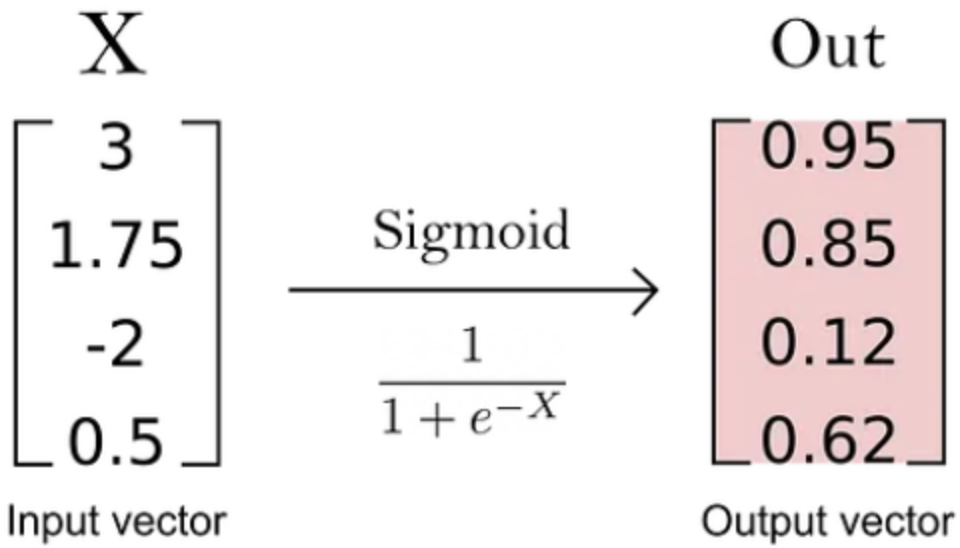
- the gradient of the cross entropy loss function with regard to $\boldsymbol{w}_k$ is given by:

$$\nabla_{\boldsymbol{w}_k}\mathcal{L}(\boldsymbol{W}) = \frac{1}{n}\boldsymbol{X}^\top(\hat{\boldsymbol{p}}_k - \boldsymbol{y}_k) = \frac{1}{n}\boldsymbol{X}^\top(\text{softmax}(\boldsymbol{o}(\boldsymbol{x}))_k - \boldsymbol{y}_k).$$

Sigmoid

2 classes

out = P(Y=class1|X)

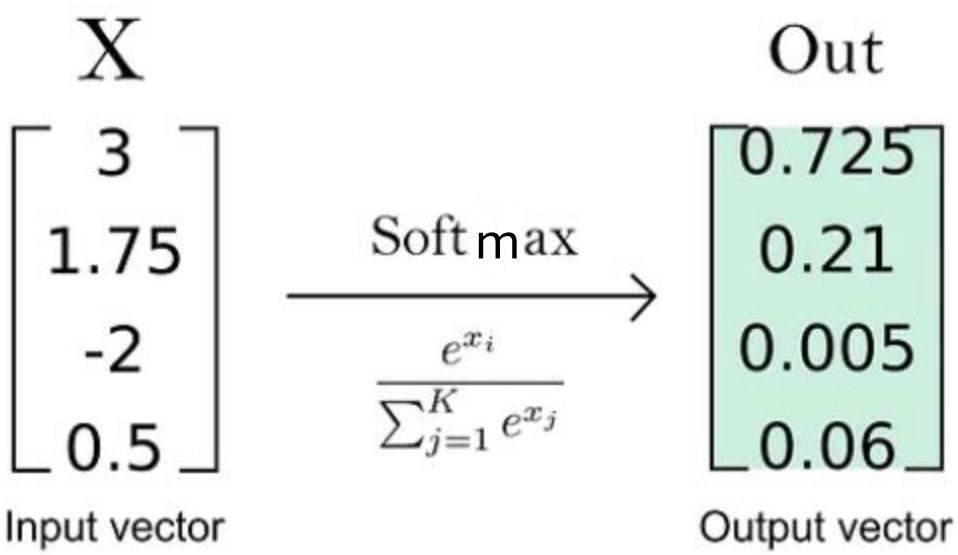


Used for logistic regression

Softmax

k>2 classes

$$\text{out} = \begin{bmatrix} P(Y=class1|X) \\ P(Y=class2|X) \\ P(Y=class3|X) \\ \vdots \\ P(Y=classk|X) \end{bmatrix}$$



Used for softmax regression

Code part

- Let's use softmax regression to classify the iris flowers into the 3 classes:

Switch from logistic to softmax regression

A variant of stochastic gradient descent

Used for l_2 regularization

```
softmax_reg = LogisticRegression(max_iter=1000, multi_class="multinomial", solver="lbfgs", C=10, random_state=42)
softmax_reg.fit(X_train, y_train)
softmax_reg.score(X_test, y_test)
```

```
0.9736842105263158
```

our model correctly classified 97.3% of the examples in the test set!

- Again, we can use the predict() method to apply the trained model on the first example in the test set:

```
softmax_reg.predict([X_test[0]])
```

```
array([2])
```

The classes are the original ones: 0 - setosa, 1 - versicolor, and 2 - virginica. Thus, the model thinks that the first example in the test set is an iris virginica !

- We can again inspect the probabilities the model assigned to the three classes, using the predict_proba() method:

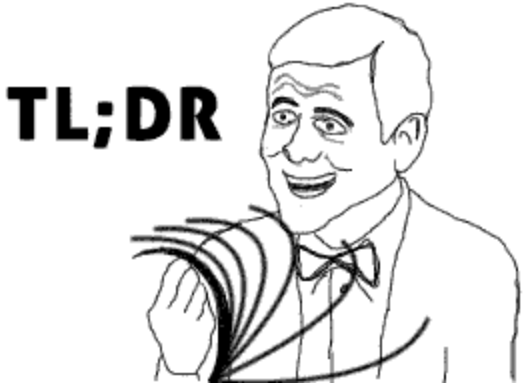
```
softmax_reg.predict_proba([X_test[0]])
```

```
array([[8.71083042e-08, 2.39654879e-03, 9.97603364e-01]])
```



This model is 99.7% certain that the first example in the test set is an iris virginica. The first two probabilities correspond to the setosa and versicolor classes, respectively.

We can verify that the three probabilities add up to 1, meaning that the model outputs a probability distribution over the three classes.



- **Linear Regression:** Learn a straight-line relationship between inputs and outputs.
- **Gradient Descent:** Iteratively adjust weights to minimize the error.
- **Polynomial Regression:** Extend linear regression to capture curved relationships.
- **Regularized Models (Ridge/Lasso):** Prevent overfitting by penalizing large weights.
- **Logistic Regression:** Turn regression into binary classification using the sigmoid function.
- **Softmax Regression:** Generalize logistic regression to handle multiple classes.

Exercises

Laboratory 2 – Exercises

1. Use the following dataset to generate a regression dataset:
`X = (2 - 3*np.random.normal(0,1,200)).reshape(200,1)`
`y = 0.5*X^3 - 2*X^2 + X + np.random.normal(-3,3,200).reshape(-1,1).`
Apply polynomial regression algorithm and plot the predictions of the model for `X_test = np.linspace(-5, 10, 200).reshape(200, 1)`.

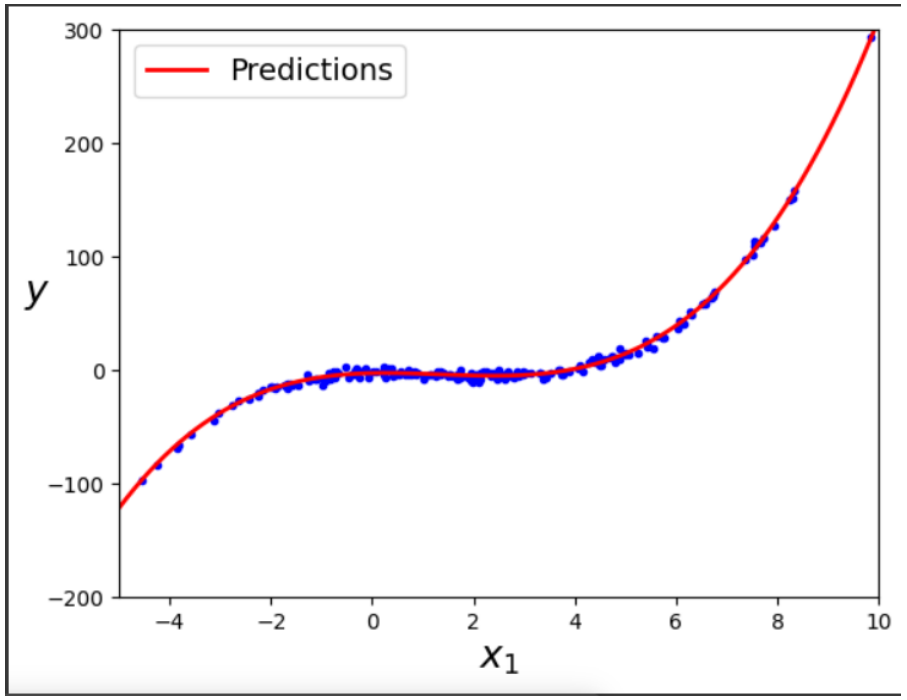
2. Use the following dataset to generate a regression dataset:
`X = 3·np.random.rand(150, 1)`
`y = -2 + 3·X + np.random.randn(150, 1).`
Perform ridge regression using mini-batch gradient descent. Use a learning rate of 0.06, 15 epochs and a batch size of 10. Set λ to 0.01. Plot the prediction of the model and print the model weights.

3. Use the *digits* dataset in order to apply the softmax regression algorithm. Load the dataset by calling the `load_digits` function. Split the data such that the test split is 35% of the initial dataset. Print the shapes of training and testing dataset. Train the algorithm for 700 epochs, use the "lbfgs" solver and set the hyperparameter C to 5. After the model is trained, print the test set accuracy. Also predict and print the probabilities that the model assigned to the ten classes for the second example in the test set.

Solutions

Exercise 1

1. Use the following dataset to generate a regression dataset:
`X = (2 - 3*np.random.normal(0,1,200)).reshape(200,1)`
`y = 0.5*X^3 - 2*X^2 + X + np.random.normal(-3,3,200).reshape(-1,1).`
Apply polynomial regression algorithm and plot the predictions of the model for `X_test = np.linspace(-5, 10, 200).reshape(200, 1)`.




```python
import matplotlib.pyplot as plt
import numpy as np
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

# Generating the dataset
# X is generated as a normally distributed random value scaled and shifted.
X = (2-3* np.random.normal(0, 1,200)).reshape(200,1)
# y is generated as a polynomial combination of X with added random noise.
y = 0.5 * X**3 - 2*X**2 + X + np.random.normal(-3,3,200).reshape(-1,1)

# Creating polynomial features with a degree of 3
# PolynomialFeatures will add additional polynomial terms (like X^2, X^3) to the dataset
poly_features = PolynomialFeatures(degree=3, include_bias=False)
X_poly = poly_features.fit_transform(X) # Transforming the original X to include polynomial features


# Fitting the linear regression model on the polynomial features
lin_reg = LinearRegression() # Initializing linear regression
lin_reg.fit(X_poly, y)  # Fitting the model with polynomial features and target variable y
lin_reg.intercept_, lin_reg.coef_ # Getting the intercept and coefficients of the model

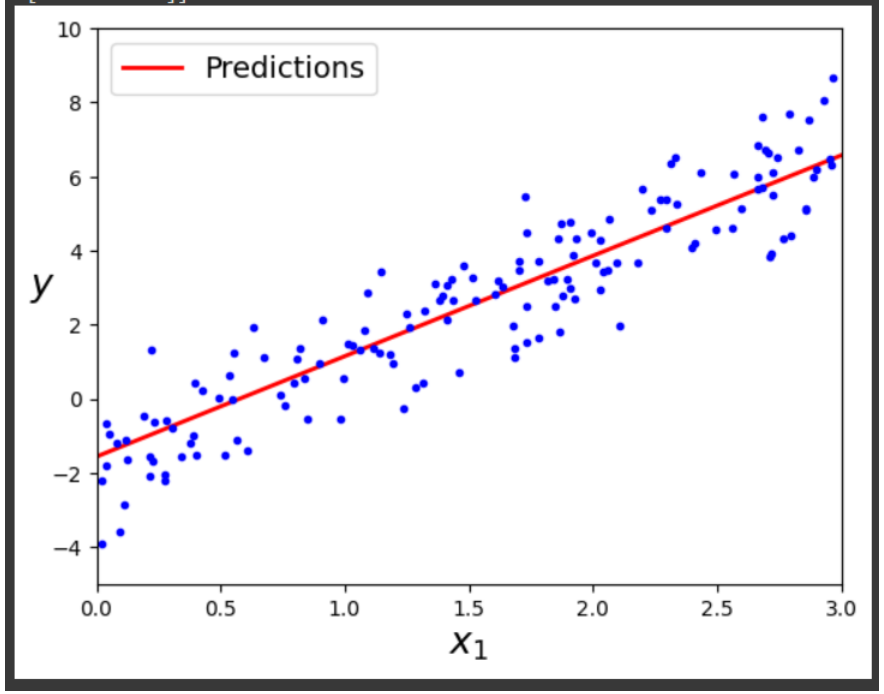
# Generating test data for predictions
X_test = np.linspace(-5, 10, 200).reshape(200, 1) # Generating 200 points between -5 and 10 for testing
X_test_poly = poly_features.transform(X_test) # Transforming the test data to polynomial features
y_test = lin_reg.predict(X_test_poly) # Predicting the values for test data

# Plotting the original data points and the regression line
plt.plot(X, y, "b.") # Plotting the original data points in blue
plt.plot(X_test, y_test, "r-", linewidth=2, label="Predictions") # Plotting the predictions in red
plt.xlabel("$x_1$", fontsize=18)
plt.ylabel("$y$", rotation=0, fontsize=18)
plt.legend(loc="upper left", fontsize=14)
plt.axis([-10, 10, -50, 100])
plt.show()
```

Exercise 2

2. Use the following dataset to generate a regression dataset:
`X = 3·np.random.rand(150, 1)`
`y = -2 + 3·X + np.random.randn(150, 1).`
Perform ridge regression using mini-batch gradient descent. Use a learning rate of 0.06, 15 epochs and a batch size of 10. Set λ to 0.01. Plot the prediction of the model and print the model weights.

```
The final model weights:
 [[-1.56594059]
 [ 2.71109173]]
```




```python
# Defining parameters for Ridge Regression using mini-batch gradient descent
eta = 0.06 # Learning rate, controls the step size for each iteration of gradient descent
n_epochs = 15 # Number of epochs, the number of times the entire dataset is passed through the model
batch_size = 10 # Batch size, number of samples per batch for each iteration
n = 150 # Number of data points
alpha = 0.01 # Regularization parameter (lambda) for Ridge Regression to prevent overfitting

# Generating the dataset
X = 3* np.random.rand(n,1) # Creating X with 150 random values scaled by 3
y = -2 + 3*X + np.random.randn(n,1) # Generating y as a linear function of X with added noise

# Initializing weights randomly
w = np.random.randn(2,1)  # Weights for the model, initialized randomly
X_1 = np.c_[np.ones((n, 1)), X] # Adding a bias term (column of ones) to X

for epoch in range(n_epochs):
    shuffled_indices = np.random.permutation(n)
    X_1_shuffled = X_1[shuffled_indices]
    y_shuffled = y[shuffled_indices]
    for i in range(0, n, batch_size):
        xi = X_1_shuffled[i:i+batch_size]
        yi = y_shuffled[i:i+batch_size]
        # Calculating the gradient (includes Ridge regularization term `alpha * w`)
        gradients = 1/batch_size * xi.T.dot(xi.dot(w) - yi) + alpha*w
        # Updating the weights using the calculated gradient and learning rate
        w = w - eta * gradients

# Printing final model weights
print("The final model weights: \n", w)

# Generating test data for predictions
X_test = np.linspace(0, 3, n).reshape(n, 1) # Generating 150 points between 0 and 3 for testing
X_test_1 = np.c_[np.ones((n, 1)), X_test]  # Adding bias term to new X values
y_test = X_test_1.dot(w) # Predicting y values using the learned weights

# Plotting original data points and predictions
plt.plot(X_test, y_test, "r-", linewidth=2, label="Predictions")
plt.plot(X, y, "b.")
plt.xlabel("$x_1$", fontsize=18)
plt.ylabel("$y$", rotation=0, fontsize=18)
plt.legend(loc="upper left", fontsize=14)
plt.axis([0, 3, -5, 10])
plt.show()
```

why this shape for w? ☺

why this formula? ☺

$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w})$

Exercise 3

3. Use the *digits* dataset in order to apply the softmax regression algorithm. Load the dataset by calling the `load_digits` function. Split the data such that the test split is 35% of the initial dataset. Print the shapes of training and testing dataset. Train the algorithm for 700 epochs, use the "lbfgs" solver and set the hyperparameter C to 5. After the model is trained, print the test set accuracy. Also predict and print the probabilities that the model assigned to the ten classes for the second example in the test set.

NOTE: it's always good to print and plot some examples to see how the data looks like and to create an idea about it. We can also use in this example `DESCR` attribute to read a description of the data.

```
from sklearn.linear_model import LogisticRegression
from sklearn import datasets
from sklearn.model_selection import train_test_split

digits_dataset = datasets.load_digits()

X_train, X_test, y_train, y_test = train_test_split(digits_dataset['data'],digits_dataset['target'],test_size=0.35,random_state=0)
```

```
print(X_train.shape)
print(y_train.shape)
print(X_test.shape)
print(y_test.shape)
```

```
(1168, 64)
(1168,)
(629, 64)
(629,)
```

```
softmax_reg = LogisticRegression(max_iter=700, multi_class="multinomial", solver="lbfgs", C=5, random_state=42)
softmax_reg.fit(X_train, y_train)
softmax_reg.score(X_test, y_test)
```

```
0.9523052464228935
```

```
softmax_reg.predict_proba([X_test[1]])
```

```
array([[1.89725693e-10, 1.80624822e-07, 3.84549572e-07, 3.03203077e-15,
        8.40304491e-04, 1.97523162e-07, 1.05764988e-09, 1.40771390e-12,
        9.99158931e-01, 6.31701135e-10]])
```

digits dataset contains images of hand-written digits: 10 classes where each class refers to a digit

the model predicts that the second example in the test set is the digit 8

```
print(digits_dataset.DESCR)
```

```
.. _digits_dataset:

Optical recognition of handwritten digits dataset
--------------------------------------------------

**Data Set Characteristics:**

:Number of Instances: 1797
:Number of Attributes: 64
:Attribute Information: 8x8 image of integer pixels in the range 0..16.
:Missing Attribute Values: None
:Creator: E. Alpaydin (alpaydin '@' boun.edu.tr)
:Date: July; 1998

This is a copy of the test set of the UCI ML hand-written digits datasets
https://archive.ics.uci.edu/ml/datasets/Optical+Recognition+of+Handwritten+Digits

The data set contains images of hand-written digits: 10 classes where
each class refers to a digit.

Preprocessing programs made available by NIST were used to extract
normalized bitmaps of handwritten digits from a preprinted form. From a
total of 43 people, 30 contributed to the training set and different 13
to the test set. 32x32 bitmaps are divided into nonoverlapping blocks of
4x4 and the number of on pixels are counted in each block. This generates
an input matrix of 8x8 where each element is an integer in the range
0..16. This reduces dimensionality and gives invariance to small
distortions.

For info on NIST preprocessing routines, see M. D. Garris, J. L. Blue, G.
T. Candela, D. L. Dimmick, J. Geist, P. J. Grother, S. A. Janet, and C.
L. Wilson, NIST Form-Based Handprint Recognition System, NISTIR 5469,
1994.

.. dropdown:: References

  - C. Kaynak (1995) Methods of Combining Multiple Classifiers and Their
    Applications to Handwritten Digit Recognition, MSc Thesis, Institute of
    Graduate Studies in Science and Engineering, Bogazici University.
```

```
X_test[1].shape
```

```
(64,)
```

```
y_test[1]
```

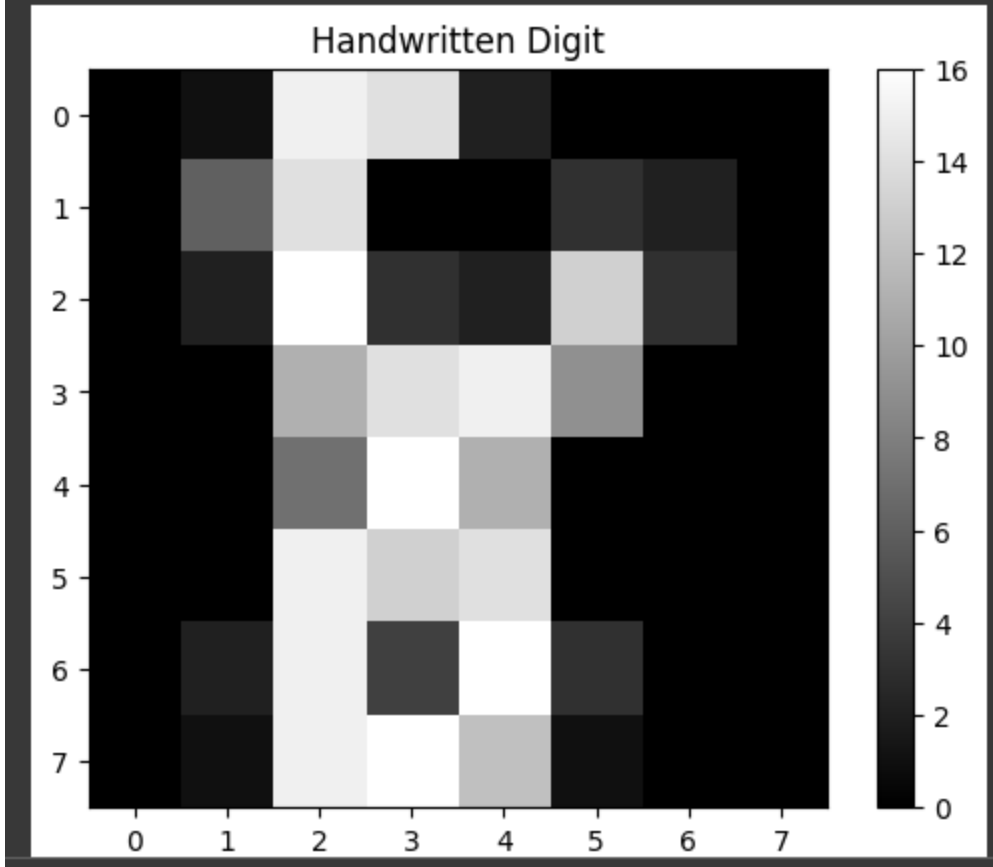
```
8
```

```
import numpy as np
import matplotlib.pyplot as plt

digit_array = X_test[1]

digit_image = digit_array.reshape((8, 8))

# Plot the image
plt.imshow(digit_image, cmap='gray', interpolation='nearest')
plt.colorbar()
plt.title('Handwritten Digit')
plt.show()
```



Thank you !